🧭 Overview

🧭RHOAI – MCP Asset Management

Last Updated: 2025-08-26

🎯 Purpose & Vision

Modern AI platforms like Llama-Stack and LangGraph increasingly rely on **MCP (Model Context Protocol) servers** to integrate with external tools, systems, and data. Organizations, however, struggle to govern these servers across teams, environments, and lifecycles.

OpenShift AI – MCP Asset Management provides a unified system to **discover, ingest, certify, govern, and deliver MCP servers** safely into enterprise environments. It encompasses:

- **Registry** – the authoritative system of record for all MCP servers, tracking versions, metadata, attestations, and certification status
- **Catalog** – a curated, policy-aware view of enterprise-approved assets
- **Deployments** – namespace-level visibility into which assets are in use
- **AI Available Assets (AAA)** – the consumption surface for AI Engineers, showing only what they are entitled to use
- **MCP Gateway/Proxy** – the connectivity layer that brokers secure runtime access to MCP servers within agent workflows and external integrations

Together, these capabilities deliver **end-to-end lifecycle control**: from sources and ingestion, through quarantine and governance, to curated catalogs, consumption, and runtime use. Designed for enterprise scale, the system includes **security scans, provenance checks, RBAC access control, GitOps integration, and gateway-mediated serving**—ensuring MCP services are trusted, observable, and consumable across OpenShift AI.

🛣️ Roadmap Snapshot - Estimated

Q4 2025 (3.0 GA – Nov 13)	Q1 2026	Q2 2026	H2 2026+

RHOAI 3.0 GA (Models Registry, MCP ConfigMap integration, AAA v1)	**Catalog UX Enhancements** – RBAC-aware views across Models & MCPs	**Unified Deployments Tab** – namespace visibility across asset types	**Agent/Guardrail Registries** (initial tech preview)
MCP ingestion via ConfigMap (playground only)	**Quarantine pipeline MVP** – SBOM, CVE scans, policy tags	**MCP Registry tech preview** – source of truth with lifecycle states	**Full AI Gateway (multi-protocol)**
Available AI Assets (AAA) – AI Engineer consumption surface	**Operator delivery** for MCP Registry	**Catalog promotion workflows** – approve/reject, risk badges	**Semantic routing + caching** (AI Gateway Phase 3)
Model Registry GA (KServe backend)	**Initial MCP Gateway** integration with Registry & Catalog	**Offline mirroring / sync UX**	**Agent-as-a-Service** + multi-tenant MCPaaS

👥 Personas

- **Platform Engineers / Managers:** Own registration, validation, certification, and catalog curation of MCP Servers; manage deployments
- **AI Engineers / App Developers:**
  - Consume certified assets via UI/APIs
  - Bind MCPs into agents/workflows
  - **Design/code new MCP Servers** for submission into the Registry pipeline
- **Business & Department Leads:** Consume assets indirectly via Agents, dashboards, and approved applications

---

🧩 Problems We Solve

Platform Admin Pain Points	AI Engineer Friction	MCP Asset Management Benefits

Wild West of 3rd-party MCP servers with unknown risk	Inconsistent tool wiring per agent	**Quarantine + validation pipeline** ensures only trusted MCPs enter the ecosystem
Hard to enforce upgrades/versioning across teams	Frequent breaks from version drift	**Semantic versioning, upgrade alerts, and lifecycle governance** prevent drift
Disconnected environments with limited visibility	Manual image copying & stale configs	**Offline sync + OCI/image export** keep environments current
No RBAC-based visibility or control	Lack of clarity on tool capabilities	**Role-based Catalog + AAA filtering, rich metadata UI** ensure clarity and control
Runtime security risks from external MCPs (e.g., prompt injection, data leakage)	Limited trust in remote services	**MCP Gateway with centralized guardrails** provides runtime protection, policy enforcement, and observability

---

🏗️ Core Concepts

Term	Description
Registry *(Platform Engineer)*	The authoritative system of record for all MCP servers. Records identity, versions, metadata, attestations, and certification states. Includes quarantine, scanning, and promotion workflows before certification.
Catalog *(Platform Engineer)*	A curated, read-only view of **certified and approved** MCP servers. Derived from the Registry and governed by RBAC/policy to ensure enterprise readiness.

MCP Gateway *(Platform/Networking Teams)*	The runtime connectivity and control layer. Provides routing, policy enforcement, guardrails, and observability for MCP traffic. Integrates with Registry/Catalog for discovery and ensures secure consumption of remote/internal MCPs. Serves as the foundation for the broader AI Gateway.
AI Available Assets (AAA) *(AI Engineer)*	The consumption surface for AI Engineers. A filtered list of assets (MCPs, Models, and in the future Agents/Guardrails/Knowledge Sources) that an engineer is entitled to use in their namespace. Derived from the Catalog and scoped by RBAC.
External Sources	Upstream origins such as GitHub, GitLab, DockerHub/Quay, Smithery, or Anthropic Registry. Used as **sources** for ingestion into the Registry pipeline.
Quarantine Zone	Secure staging area where discovered MCP servers undergo validation: containerization (if needed), SBOM/CVE/license scans, provenance checks, and conformance testing. Prevents risky components from being promoted until validated.
MCP Server Metadata	Standardized metadata (JSON schema) describing each server's identity, supported MCP spec versions, endpoints, security posture, maturity, license, tools, and attestation results.

🔍 Metadata & API Surface

Each MCP server is described using a standardized document that includes:

- Required: id, name, version, description, toolList, mcpSpecVersion, license, connections[], createdAt, updatedAt
- Optional: authMethod, x-certificationStatus, x-securityScan, x-maturityLevel, etc.

APIs and features:

- **GraphQL & REST**: Upload, promote, deprecate, and search
- **Webhooks & GitOps feeds**: Emit SBOMs, scan results, and upgrade notices
- **MCP CLI/SDK**: Resolve and pull servers via mcp://registry/name@version

---

✅ Security & Certification Flow

1. **Import & Quarantine** MCP servers (e.g. GitHub, Kubernetes) are imported manually or via sync, scanned for CVEs and malware, and validated before further steps.

2. **Scan & Metadata Validation** Registry extracts and validates metadata.json. Clair-based CVE scans and optional malware checks are executed in isolation.

3. **Promotion to Catalog** Admins review security reports and metadata. Certified servers are promoted for controlled consumption via Catalog.

4. **Consumption & Integration** AI Engineers browse their Available Assets, selecting approved MCP servers for agent design. CLI/SDK or UI snippets allow easy integration.

5. **Upgrade & Deprecation Lifecycle** Registry monitors version compatibility, surfaces upgrade notices, and eventually deprecates outdated or revoked entries.

🌐 Ecosystem & External Integration

Source	Description
Docker MCP Catalog	Over 100 curated MCP servers with OAuth vault and metadata. Mirrored via registry sync.
Hugging Face Spaces	Plugin-style MCP services that integrate AI tooling; treated as external registries.
Partner Registries	Auto-certification pathways under development via Red Hat Partner Connect.

Sample Certified Servers:

- **GitHub** – Project metadata, PR workflows, code access
- **Kubernetes/OpenShift** – Resource inspection and cluster control
- **Email/G-Suite** – Enterprise messaging and scheduling
- **Context7/Docs** – External documentation readers and summarizers

---

🧪 UX Workflow (Demo Highlights)

1. **Catalog View**: Filtered grid of certified MCP servers with categories and verification status.
2. **Details Drawer**: Expand to see version history, security scan results, and connection info.
3. **Import Wizard**: Add new MCP servers by URL; validate and move into quarantine.
4. **Scan Status**: Visual feedback during scans; blocks promotion until passing.
5. **Agent Tooling**: YAML snippets and auto-wiring from Available Assets into agent design.

🆚 Competitive Differentiation

Feature	Docker MCP Catalog	Hugging Face Tool Hub	MCP Registry
Container security scanning	✅ (docker scan, Trivy)	❌	✅ (Clair, Trivy supported)
Malware / pickle / secret scan	⚠️ Partial (container-level)	✅ (pickle, secrets, ClamAV)	✅ (malware scan planned)
Access control & RBAC	✅ (Docker Teams/orgs)	✅ (org tokens, 2FA, SSO)	✅ (OpenShift RBAC native)
Quarantine & promotion gating	✅ (basic pipeline)	❌	✅ (multi-stage gating)
Air-gapped sync support	Roadmap	❌	✅ (sync export + registry)

Feature	Docker MCP Catalog	Hugging Face Tool Hub	MCP Registry
GitOps hooks	❌	❌	✅ (webhooks, signed feeds)
CLI / SDK support	✅ (docker mcp CLI)	⚠️ Partial (HF Hub APIs)	✅ (mcp:// + SDKs)

Main Areas

🏢 Main Functional Areas

Last Updated: 2025-08-26

📚 MCP Catalog – Discovery & Consumption

The **Catalog** remains a **separate architectural component** used to expose a curated, enterprise-approved list of MCP Servers. It is not a governance system but a **reflection of Registry-vetted assets**.

🔍 Platform Engineer Workflow

- Browse the Catalog for **certified MCP Servers**
- Catalog entries flow from the Registry based on **policy + RBAC filtering**
- Consumption occurs via OperatorHub packages, Helm charts, or internal provisioning tools

🔐 RBAC + Policy Enforcement

- Views are always RBAC-scoped
- Surfaces only **certified + approved** MCP Servers
- External and internal MCP Servers appear uniformly after certification
- Long-term, Catalog and Registry views can also integrate into Developer Hub plugins

---

🔒 MCP Registry – Governance & Lifecycle

The **MCP Registry** is the **system of record** for all MCP Servers, whether internal or external. It governs ingestion, lifecycle, security, and certification workflows.

🛠️ Ingestion & Lifecycle

▪ Internally Hosted MCP Servers

- Sources include GitHub/GitLab repos, Operators, or tarball submissions
- Non-containerized MCPs are containerized during intake
- Enter **Quarantine** for SBOM generation, CVE scanning, provenance checks
- Promotion to Registry occurs only after passing certification and policy gates

▪ **Externally Hosted MCP Servers**

- Registered by **metadata-only** (no image pull)
- Full `server.json` metadata required
- Governed via policy tags (e.g., maturity, certification, risk)

✅ Core Registry Features

- JSON schema includes fields for `x-certificationStatus`, `x-securityScan`, `x-maturityLevel`, and `version` to support traceability
- OAuth support per the **MCP 2025-06-18 Authorization Spec**
- Secure object storage for vetted internal MCP artifacts
- Disconnected OpenShift environments supported via **offline mirroring / sync** strategies (post-MVP)
- Registry integrates with external upstreams (e.g., GitHub, Hugging Face, partner registries) but applies **quarantine + certification** before surfacing in the Catalog

---

🌉 MCP Gateway – Connectivity & Control

The **MCP Gateway** is the first phase of the broader **AI Gateway**, acting as the intelligent, multi-protocol access layer for MCP traffic. It provides the secure runtime path between agents, models, and tools.

🌐 Role in the Ecosystem

- Serves as the **entry point for MCP workloads**, regardless of source or hosting environment
- Provides **routing intelligence** between agents and MCP servers
- Enforces **policy, authentication, authorization, and rate limiting** on a per-tool basis
- Offers **observability (logs, metrics, audit events)** for all MCP traffic
- Scales to support multiple MCP servers with **stateful session management**

🚀 Strategic Objectives

- Deliver **Phase 1 MCP Gateway** quickly to handle MCP traffic end-to-end
- Integrate directly with the **MCP Registry and Catalog** for server discovery and tool aggregation
- Build on OpenShift Gateway API foundations (Envoy/Istio)
- Establish a foundation for multi-protocol expansion (A2A, emerging AI standards)

🔑 Why It Matters

- **Enterprise control:** Ensures MCP access is governed, observable, and auditable
- **Integration layer:** Provides the runtime bridge between governance (Registry/Catalog) and consumption (AAA, Studio)
- **Future-proof:** Prepares OpenShift AI for additional AI protocols and semantic routing

---

📖 AI Available Assets (AAA – Consumption Surface)

The **AI Available Assets (AAA)** view is where AI Engineers discover and consume assets that they are entitled to use in their namespace.

- **Scope:** RBAC- and namespace-filtered view of assets made available by Platform Engineers
- **Includes:** Certified MCP Servers, approved Models, and (future) Agents, Guardrails, and Knowledge Sources
- **Access Points:** UI (card or table layouts) and `/resolve` API
- **Usage:** AI Engineers select from the list to wire assets into agent builds or workflows without needing to manage ingestion/registry themselves
- **Outcome:** Provides a clear, secure boundary — Engineers only see and use assets granted to them builds or workflows**
- Engineers decide which resources/tools to include in their agent

---

🧩 Unified AI Asset Registry – Long-Term

Models lead the way – their Registry, Catalog, and deployment will be GA first. **MCP Servers** follow a similar trajectory, with eventual convergence into a unified registry model.

The long-term goal is **one unified AI Asset Registry**, but each asset type may leverage different backends (e.g., Models on KServe, MCPs aligned to the Anthropic Registry standard).

This ecosystem will govern:

- **Models** – ✅ GA soon
- **MCP Servers** – ✅ Following models (RHOAI v3.next)
- **Agents** – Future
- **Guardrails** – Future
- **Knowledge Sources** – Future
- **Other AI Asset Types** (e.g. prompts, policies) – Potential future

Each asset type will maintain its own **Registry + Catalog**, but all operate under **shared RBAC, policy enforcement, observability, and certification pipelines**.

🧠 Breakdown

🧭 MCP Management Strategy Summary

Last Updated: 2025-08-26

🔹 1. MCP Server Ingress & Lifecycle

Ingress → Quarantine → Registry → Catalog → Deployments → Consumable

- **Ingress Sources:**
  - **External Registries & Repos** (e.g., Anthropic, GitHub, Docker Hub, Smithery)
  - **Internal Repos** (enterprise/private MCP server codebases)
  - **Operators** (OpenShift-delivered MCP server Operators)
- **Quarantine Stage:**
  - Imported MCPs undergo SBOM generation, CVE scanning, provenance verification, and policy tagging
  - Servers remain quarantined until approved
- **Internal Registry (Red Hat MCP Registry):**
  - Central control plane for MCP servers
  - Stores metadata (version, specVersion, security status, maturity level, etc.)
  - Enforces RBAC and lifecycle state transitions
- **Catalog (AI Hub):**
  - Enterprise-approved shortlist of certified MCP servers curated by Platform Engineers
  - Catalog entries are what **Platform Engineers can grant to AI Engineers** for namespace use
- **Deployments (AI Hub):**
  - Deployment tab shows which assets (Models, MCPs, etc.) have been deployed into specific namespaces
- **AI Available Assets (AAA):**
  - The workspace-level surface where **AI Engineers see assets they have rights to consume**
  - Lists are filtered by RBAC + namespace permissions

🔹 2. MCPaaS (MCP-as-a-Service)

- **Motivation:** Simplify and scale MCP hosting and lifecycle management
- **Offering:**
  - On-demand MCP server provisioning
  - Centralized auditing of usage, versions, and configurations
  - Multi-tenant departmental/agency use cases
  - Integration with **Models-as-a-Service (MaaS)** for agent workflows and lifecycle governance

---

🔹 3. Design & Consumption Patterns

- **Registries decoupled from LlamaStack** but integrable for agent orchestration
- **Workflow:**

  1. MCP pulled from **External Registry, Internal Repo, or Operator**
  2. Quarantined → scanned, tagged, certified
  3. Promoted into **Internal Registry**
  4. Visible in **Catalog** (AI Hub) based on RBAC
  5. Deployable into namespaces
  6. Discoverable by AI Engineers via **AI Available Assets (AAA)**

🔹 4. AI Hub View Structure

- **Catalog Tabs:** Enterprise-approved assets by type (Models, MCP Servers, Agents, Guardrails, Knowledge Sources)
- **Registry Tabs:** Metadata, certification state, and lifecycle tracking for each asset type
- **Deployments Tabs:** Show where assets are deployed and bound to namespaces
- **AAA (AI Available Assets):** Separate view for AI Engineers, filtered by namespace + RBAC rights

---

🔹 5. Personas & Roles

- **Platform Engineers / Managers:** Own registration, validation, certification, and catalog curation of MCP Servers; manage deployments
- **AI Engineers / App Developers:**
    - Consume certified assets via UI/APIs
    - Bind MCPs into agents/workflows
    - **Design/code new MCP Servers** for submission into the Registry pipeline
- **Business & Department Leads:** Consume assets indirectly via Agents, dashboards, and approved applications

---

🔹 6. Northbound Objectives

- Deliver **Models first (GA via KServe backend)**, MCPs next (**Anthropic registry-aligned backend**), then Agents, Guardrails, Knowledge Sources
- Support RBAC, multi-tenancy, and offline/disconnected environments (sync/mirroring)
- Provide **single-pane-of-glass** asset management in AI Hub + **consumption rights surface in AAA**
- Align with Operator-based delivery (Models, MCPs, LlamaStack) for seamless OpenShift installation and upgrades

♻ MCP Server Lifecycle

♻️ MCP Server Lifecycle

Last Updated: 2025-08-26

This lifecycle provides a standardized process for introducing, validating, approving, and managing MCP servers across Red Hat environments. It reflects current MVP constraints while preparing for long-term registry, catalog, and gateway integration.

---

1. Sources (Ingress)

- Internal development repos (products/teams)
- Third-party/external repositories
- MCPs bundled with Operators or sidecars
- **MVP approach:** project/cluster-level approved list (config-map); expand to Registry + Catalog later

---

2. Intake & Normalization

- Extract metadata: server name, endpoints, tools, required environment variables/secrets
- Cache server details for discovery (shown even if server is offline)
- Refresh metadata automatically when server becomes reachable

3. Build & Packaging

- All internally developed MCP servers must be **containerized**
- Non-container/native deployments will not be admitted into product environments

4. Security & Compliance Gates (Toolhive)

- Generate SBOMs and provenance data
- Perform vulnerability and dependency scans
- Enforce organizational policies (CVEs, licenses, base image restrictions)
- Create signed attestations as evidence for promotion

5. Registry (Source of Truth)

- Register candidate MCP servers in the **MCP Registry** (Anthropic-aligned model)
- Store identities, versions, metadata, health checks, attestations, and approval status
- Maintain lifecycle history (promotions, deprecations, retirements)

6. Approval Workflow

- Formal enterprise approval process before publication
- Approved MCPs become visible to downstream consumers
- Approval is **separate from runtime enforcement**

---

7. Catalog (Discovery)

- Publish approved MCPs into Catalogs for user discovery (e.g., AI Hub "Available Assets")
- MVP: Config-map–based lists; future: registry-driven Catalogs
- Catalog is curated for visibility, **not enforcement**

---

8. Gateway (Policy & Enforcement)

- AI Gateway enforces:
    - Authentication & authorization
    - Egress control and rate limiting
    - Logging and audit
    - Runtime tool filtering and guardrails

9. Consumption (Playgrounds & Agents)

- AI engineers select MCPs from the approved list
- Configure and export connection settings (e.g., env vars)
- Health checks determine MCP availability and tool access

10. Runtime Health & Telemetry

- Continuous health monitoring (basic in MVP)
- Report availability, last-seen tool list, and telemetry data
- Surface degraded or stale MCPs for re-evaluation

11. Change Management

- New MCP versions go through the same gates as initial intake
- Policies control promotions, rollbacks, and approvals
- Registry preserves full lifecycle audit trail

12. Deprecation & Retirement

- Deprecated MCPs marked in Registry/Catalog
- Blocked from new attachments via Gateway enforcement
- Retired MCPs removed from discovery and usage

Lifecycle States & Gates

States: Draft → Candidate → Verified → Approved → Published → Deprecated → Retired

Gates: Security (scan/sign) · Compliance (license/policy) · Platform (reachability/egress) · Product (UX/tenant fit) · Operations (SLOs/monitoring)

Where Stacklok Toolhive fits (high-level)

Toolhive is a great backbone for **software supply-chain** hygiene and **policy-driven promotion** of MCP servers. In this lifecycle it would:

1. **Source verification & provenance:** Link PRs/commits to builds, produce SLSA-style provenance, and generate SBOMs for MCP images/packages.

2. **Vuln & policy scanning:** Scan images/deps; enforce org policies (CVSS thresholds, license allowlists, disallowed base images, banned APIs).

3. **Attestations & signing:** Create/store cryptographic attestations (scan results, provenance) and sign artifacts. These become *evidence* for promotion to "Approved."

4. **Promotion workflows:** Implement *Dev → Staging → Approved* lanes with policy checks; emit webhook/events to update **MCP Registry** records and flip visibility in **Catalog** when a build passes.

5. **Continuous compliance:** Re-score MCPs as CVEs and policies change; auto-open "Re-verify" tasks or auto-deprecate if risk exceeds thresholds.

6. **Gateway policy hooks (optional):** Feed signed metadata to the Gateway (e.g., only route to MCPs whose image digest + attestation set is valid), enabling runtime enforcement to match build-time truth.

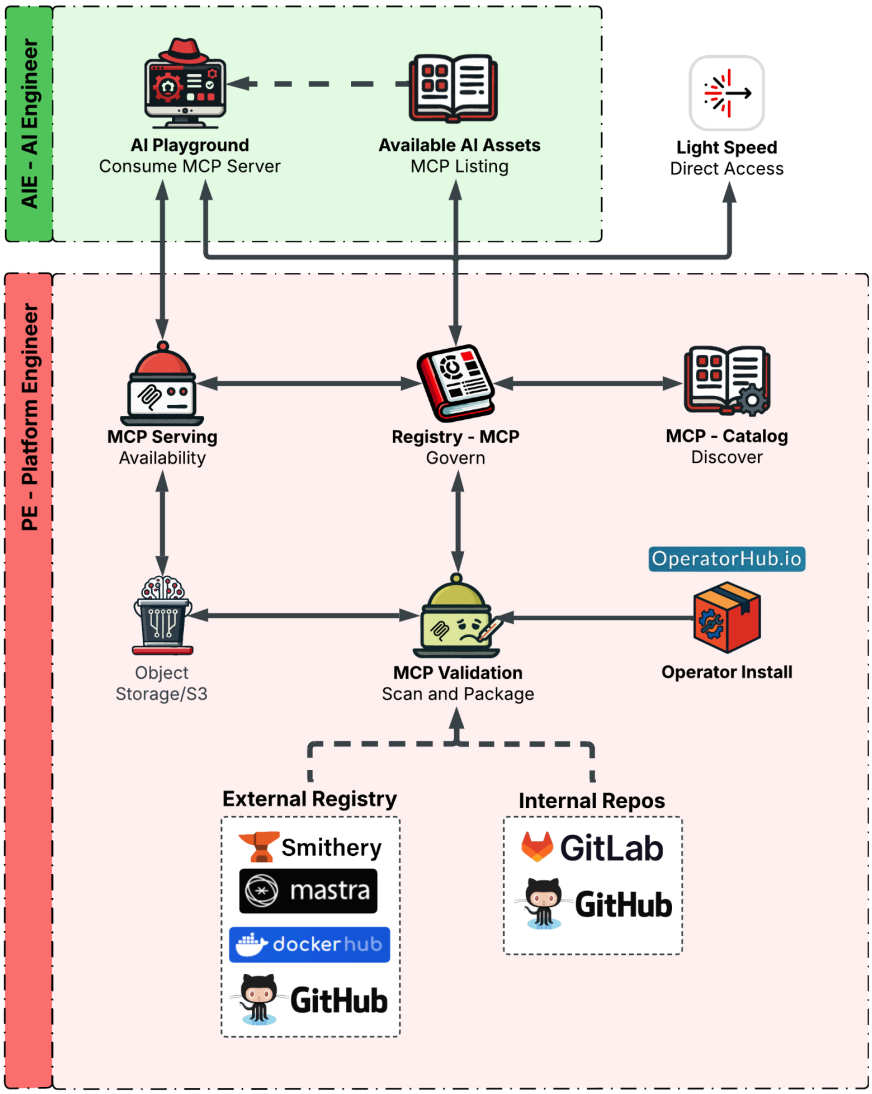
Key Alignment Notes

- **MVP path:** Config-map based approved list + basic health checks → evolve to Registry + Catalog + Gateway
- **Containerized only:** MCP servers must be containerized for admission
- **Tenant/namespace model:** Namespace-scoped approved lists now; multi-tenant model later

🔷 MCP Asset Flow

🔄 MCP Asset Flow

Last Updated: 2025-08-26



🌐 1. Sources (Discover)

👤 Persona: Platform Engineer

The entry point for ingesting potential MCP Servers.

- **Sources**: External Registries (GitHub, DockerHub, Smithery, Anthropic), Internal Repos (GitLab, private repos), Operators (OperatorHub), Remote Endpoints (already-hosted MCPs)
- **Output**: Candidate MCP Servers discovered with minimal metadata — not yet trusted or consumable

---

🧪 2. Quarantine & Validation (Security)

👤 Persona: Platform Engineer

Untrusted MCP Servers are isolated and validated before Registry inclusion.

- **Trigger**: Candidate entry pulled from Sources into Quarantine
- **Actions**:
  - Containerization (if needed) + metadata inspection
  - SBOM, CVE, and license scans → update x-securityScan
  - Certification state marked quarantined in metadata (x-certificationStatus)
  - Store artefacts in object storage (e.g., S3/ODF)

- **Goal**: Verify MCP spec alignment, code safety, and provenance integrity

---

🛡️ 3. Registry (Govern – Source of Truth)

👤 Persona: Platform Engineer

The authoritative system of record for MCP Servers.

- **Governance Flow**:
  - Record identity, source URIs, version, digest, and metadata at discovery
  - Track transitions: *discovered → quarantined → certified → approved/blocked → deprecated*
  - Attach attestations (e.g., SBOM, conformance results, provenance)
  - Assign certification + maturity levels (community, silver, gold)
  - Apply RBAC to scope visibility (team, BU, environment, role)

- **AI Hub Role**: Registry tab (per asset type)

📚 4. Catalog (Curated View)

👤 Persona: Platform Engineer

The **Catalog is a policy-aware view over the Registry** showing only enterprise-approved assets.

- **Function**:
  - Surfaces only certified/approved MCPs per RBAC + policy
  - Provides trust badges, risk scores, and provenance links
  - Allows one-click enable/install into namespaces
- **AI Hub Role**: Catalog tab (per asset type)

🚀 5. Deployments (Namespace Binding)

👤 Persona: Platform Engineer

Deployment of approved assets into namespaces.

- **Function**: Tracks which Models, MCPs, or other assets are deployed
- **Purpose**: Bridge between Registry-governed assets and AAA-consumable assets
- **AI Hub Role**: Deployments tab (per asset type)

📖 6. AI Available Assets (AAA – Consume)

👤 Persona: AI Engineer / App Developer

The consumption surface for AI Engineers — shows what assets they can use in their namespace.

- **Access**: RBAC-scoped browsing via AAA UI or `/resolve` API
- **Includes**: Version history, protocol support, usage docs, tool summaries
- **Used For**:
  - Selecting toolchains for agents

    - YAML-based orchestration and integration into workflows

---

🤖 7. Studio & Serving (Use)

👤 Persona: AI Engineer

Deployment and runtime use of MCP Servers inside **RHOAI GenAI Studio**.

- **Platform**: RHOAI Studio with MCP Serving backend
- **Features**:
    - Select certified MCP Servers for agent tool access
    - Auto-deploy via Serving runtime
    - Config injection + endpoint lifecycle management

⚡ 8. LightSpeed Direct Access

👤 Persona: LightSpeed & Platform Teams

- **Purpose**: LightSpeed queries the MCP Registry directly
- **Requirements**: Certified-only filtering, RBAC enforcement, metadata compliance
- **Benefits**: Direct integration into workflows without requiring UI interaction

🔑 Key Flows

🔑 Key Flows

How It Works – Flow Snapshot

- **Import & Quarantine** – Admin uploads or mirrors a server; registry validates schema and runs security scans in isolated quarantine.
- **Promotion & Certification** – Human reviewer approves passing servers, making them visible in the public Catalog.
- **Consumption (Agent Build-Time)** – AI / Agent engineers reference certified servers via mcp:// URIs; CLI resolves and pulls immutable digests.
- **Mirroring / Sync** – Policy based checks, sync trusted subsets from external catalogs (e.g., Github MCP) into quarantine for auto-promotion.
- **Upgrade Advisory** – Registry flags newer compatible versions and surfaces upgrade hints in CLI and UI.
- **Deprecation / Retirement** – Deprecated servers emit warnings and are eventually blocked from new pulls after a grace period.

Import & Quarantine Flow

Goal	Bring an MCP server into the registry while preventing unvetted code from reaching production.
Actors	Platform Admin (initiator), Registry API, Security Scanner, Quarantine DB
Steps	1. Admin calls `POST /servers/import` with a container image reference or tarball 2. Registry pulls the image, extracts `metadata.json`, sets `x-certificationStatus = "quarantined"`, generates **SBOM** and attaches to x-securityScan.sbomUrl,writes to DB 3. Scan pipeline (Clair → Syft → Grype via Tekton) and stores results under `x-securityScan` 4. Webhook fires → SecOps ticket (optional). 5. Admin notified; flow hands off to Promotion.

Key Gates	• Image pull succeeds; schema validation passes • Scan completes within SLA; Critical/High CVEs = 0 by default; licence audit passes
Artifacts & APIs	`importJobId`, server record, SBOM file; endpoints: `/import`, `/jobs/{id}`, `/servers/{uuid}`; sbom.cdx (or SPDX)

Promotion & Certification Flow

Goal	Decide whether a quarantined server becomes "certified" and visible in the Catalog.
Actors	Platform Admin (reviewer), Optional Product Owner, Registry UI/API
Steps	1. Reviewer opens server detail page or queries GraphQL 2. Examines scan summary, license, README preview, smoke-test log, CVE / licence summary from Clair + Grype 3. Clicks **Approve** → registry sets `x-certificationStatus = "certified"` 4. Promotion webhook notifies consumers; catalog search index refreshes.
Key Gates	• License is OSI-approved • Maturity level fits environment (e.g., not "experimental" for prod catalog) • Critical/High CVE count within policy
Artifacts & APIs	`/servers/{uuid}/promote`, status change event, Catalog index update

---

Consumption Flow (Agent Build-Time)

Goal	Let AI engineers pull certified servers declaratively while locking versions.
Actors	AI/Agent Engineer, MCP-Client CLI/SDK, CI or local dev box, Registry API

Steps	1. Engineer writes `mcp://registry/github@1.2.0` into agent config 2. CLI resolves URI via `/resolve?name=github&version=1.2.0` and returns image digest + metadata 3. Extract/set env vars from meta data and Local Docker/podman pulls image; build proceeds 4. CLI caches metadata for offline replay (repeat builds are faster)
Key Gates	• Registry returns HTTP 200 and `x-certificationStatus = "certified"` • If status ≠ certified, CLI aborts unless `--allow-uncertified`.
Artifacts & APIs	`/resolve`, local cache, pulled image layer digests

---

Mirroring / Sync Flow (External Registries)

Goal	Keep a subset of trusted servers, similarly to Docker's MCP Catalog or Hugging Face in sync with the on-prem registry.
Actors	Mirror Controller (operator), External Source Registry, Internal Registry
Steps	1. Admin defines a `MirrorPolicy` CRD with source URL + filter (e.g., tag=latest, maintainer=RH) 2. Controller polls external `/catalog/index.json` every N hours 3. New or updated servers go through **Import & Quarantine** flow automatically 4. If previously certified, re-promotion can be auto-approved if diff shows only patch-level change
Key Gates	• Source registry reachable • Auth token valid • Policy filter passes
Artifacts & APIs	CRDs `MirrorPolicy`, `MirrorStatus`; external & internal `/catalog`; import jobs

---

Upgrade Advisory Flow

Goal	Notify users when a safer or more recent compatible version exists.
Actors	Registry, MCP-Client, Agent Runtime
Steps	1. When a new certified version > current, registry tags the old one with `x-isOutdated = true` and `recommendedVersion` 2. Next `mcp resolve` call returns both current and recommended versions 3. CLI prints warning or CI pipeline emits "upgrade available" note 4. Engineer updates config; new build picks the fresher image
Key Gates	Semantic version comparison matches upgrade rules (e.g., same major).
Artifacts & APIs	`/servers/{uuid}`, CLI stdout message, upgrade metrics

Deprecation / Retirement Flow

Goal	Gracefully phase out servers that are insecure, unmaintained, or superseded.
Actors	Platform Admin, Registry API, MCP-Client
Steps	1. Admin sets `x-certificationStatus = "deprecated"` and adds `deprecationDate` + `replacement` fields 2. Registry UI marks entry with yellow banner; CLI emits deprecation notice 3. After deprecationDate + grace period, registry can refuse new pulls (`HTTP 410`) 4. Historical metadata remains for audit
Key Gates	• Replacement identified or risk accepted • Adequate notice period (e.g., 30 days)
Artifacts & APIs	Status change event, `/deprecated` list, 410 responses

---

How to Use These Flow Docs

1. **Developer Docs** – Drop each flow table (or convert to Mermaid sequence diagrams) in the "Architecture" section of your README/dev-portal.
2. **Operator Runbook** – Extract the *Steps* + *Key Gates* as checklist items for SRE teams.
3. **Security & Compliance** – Reference the flow tables in your SSDF / SOC 2 evidence to show supply-chain controls.

📷 Snapshot

Snapshot — why these matter

Red Hat's enterprise value isn't just "hosting MCP servers." Customers will judge us on how well we secure the entire software-supply chain, comply with fast-moving AI regulations, control costs, and guarantee ethical/observable AI. These topics therefore broaden your impact beyond registry mechanics.

Software-Supply-Chain Hardening

What & Why

- **SBOM & VEX workflows** — customers want verifiable component lineage for every MCP server we certify.
- **SLSA levels** — up-and-coming procurement check box; expect RFPs to ask whether MCP builds reach SLSA 3/4.
- **Trusted Profile Analyzer / Red Hat Trusted Software Supply Chain** tools are already in portfolio.

Starter read: Red Hat's Trusted-Supply-Chain overview (Red Hat)

Regulatory & Ethical AI Compliance

What & Why

- **EU AI Act, U.S. Executive Orders, NIST AI RMF** will impose mandatory risk classification and reporting on agentic systems. Registry metadata and tool access policies must surface "high-risk AI" disclosures.
- Understand how fairness, bias, and explainability testing fit into MCP intake.

Starter read: HBR explainer on complying with the EU AI Act (Harvard Business Review)

FinOps & Cost Governance for Agent Platforms

What & Why

- Customers will ask for quota enforcement, rightsizing guidance, and show-back dashboards.

Starter read: OpenShift cost-optimization best practices (Medium)

Isolation & Sandboxing Patterns

What & Why

- Meeting discussion highlighted quarantine/sandbox intake.
- OpenShift Sandboxed Containers (Kata) give you a productised answer; learn how to position them against regular containers and VMs.

Starter read: OpenShift Sandboxed Containers blog (Red Hat)

Multi-Tenancy & Scoped Access

What & Why

- MCP "views by namespace" depend on solid Kubernetes multi-tenancy patterns (hierarchical namespaces, NetworkPolicies, etc.).
- Study trade-offs between soft (RBAC) and hard (node/pod) isolation.

Kubernetes multi-tenancy guide (Kubernetes)

AuthN/AuthZ Extensions & Partner Tooling

What & Why

- Stacklok's **ToolHive** and similar projects externalise OAuth 2.0 + OpenID Connect for MCP servers.
- Useful to decide "build vs. integrate" for fine-grained tool gating.

Secure-by-default ToolHive post (DEV Community)

Agent-Centric Threat Modeling

What & Why

- New exploits (tool poisoning, RAG poisoning, rock-pooling) target multi-tool agents.
- Build threat-model templates and reference mitigations into intake docs.

Overview of RAG/tool-poisoning risks (Medium)

Open-Source Governance & Licensing

What & Why

- Every MCP server we ship must follow Red Hat open-source participation guidelines and dual-license hygiene.
- You'll interface with OSPO on legal review of partner servers.

Red Hat participation guidelines (Red Hat)

Observability, Tracing & SLOs for MCP Workloads

What & Why

- Large customers will require end-to-end tracing (OpenTelemetry) across agent flows.
- Know Red Hat's distributed-tracing operator and how to expose per-tool metrics.

Distributed tracing across clusters (Red Hat)

Fairness, Bias & Model Evaluation Pipelines

What & Why

- Registry "certified" badge may eventually need bias/fairness attestation for any model-backed MCP server.
- Familiarize yourself with common metrics (DI, EOD, SPD) and testing suites (Fairlearn, Aequitas).

Intro to model-fairness evaluation (Arize AI)

Optional Deep Dives (as bandwidth allows)

Topic	Why it may surface soon
Data-Residency & Sovereign Cloud	EU & APAC tenders increasingly demand evidence that agent data stays local.

Incident-Response Automation	Agents manipulating infra means new playbooks for SOC teams.
Marketplace Monetization Models	How partners will list paid MCP servers in a future Red Hat catalog.
Disaster Recovery & Backup of Registries	Auditable history of versions, signatures, & SBOMs.

How to use this list

1. **Prioritize**: Start with sections 1–4; they underpin partner conversations and upcoming design sessions.
2. **Create one-page primers**: For each topic, summarize "customer concern → Red Hat answer → open questions."
3. **Schedule brown-bags**: Bring in SME's (e.g., OSPO on licensing, FinOps team on cost).
4. **Feed discoveries** into the joint MCP+OpenShift AI strategy

🏪 MCP Catalog

🏪 MCP Catalog – Product Management Write-Up

Last Updated: 2025-10-01

Executive Summary

The **MCP Catalog** will be the governance and management layer of the Model Context Protocol (MCP) ecosystem within Red Hat OpenShift AI. It builds on the **Registry** (authoritative source of MCP servers) and complements the **Gateway** (runtime enforcement) by managing the full lifecycle, relationships, and governance of MCP entities such as tools, prompts, resources, workloads, and eventually agents.

Unlike a purely internal component, the MCP Catalog must operate in a **federated and interoperable ecosystem**, where clients may bring their own catalogs or registries (e.g., Docker MCP Catalog, the open MCP Registry, Microsoft Copilot Catalog, or Databricks Unity Catalog). Its purpose is to provide **discoverability, governance, lifecycle management, and trust** across MCP assets while integrating seamlessly into diverse customer environments.

Business Needs

- **Governance** – Enterprises need clear control over which workloads may use which MCP tools, prompts, and resources.
- **Security & Trust** – Catalogs must provide **publisher verification, moderation, and trust signals** (verified publishers, certifications, provenance) to reduce risk from untrusted connectors.
- **Operational Usability** – LLMs degrade in performance when overloaded with irrelevant or overly complex tools. The catalog ensures proper filtering, simplification, and masking.
- **Lifecycle & Compliance** – Entities must be tracked with **states (draft → review → approved → deployed)** and provide full auditability.
- **Federation & Interoperability** – Clients may bring external registries or gateways; the catalog must **federate** with external sources while supporting internal private catalogs.

- **Scalability** – Enterprises need to manage thousands of MCP assets in an organized, discoverable, and efficient manner.

Goals

- Provide a **single system of record** for MCP entities beyond servers.
- Enable **fine-grained workload governance** (workload ↔ tool/resource/prompt mappings).
- Deliver **masking, filtering, and simplification** of tools/resources for agent usability.
- Support **entity-level versioning** with coexistence, controlled rollouts, and deprecation flags.
- Ensure **federation with external registries** and **interoperability with customer-provided catalogs/gateways**.
- Provide **APIs and UI** for discoverability, audit, and administration.
- Serve as the foundation for a broader **AI Assets Catalog** (models, agents, prompts, datasets).

Core Requirements

Managed Entities

- MCP Servers (sourced from Registry)
- Tools and Resources
- Prompts
- Workloads
- Workload ↔ Tool/Resource mappings

Governance & Trust Features

- Restrict workloads to only approved tools
- Disable or revoke tool usage globally or per workload when vulnerabilities arise

- Apply **enterprise governance metadata** (ownership, compliance tags, risk classification)
- Support **publisher verification and moderation** (trust signals for catalog entries)

Filtering & Simplification

- **Tool Masking** – expose only necessary tools per workload
- **Input Simplification** – break multi-purpose tools into clearer, single-purpose variants
- **Output Filtering** – reduce responses to only relevant fields for LLMs

Lifecycle Management

- **Ingress** – MCP servers discovered, reviewed, and added to the catalog
- **Review/Approval** – entity changes validated before deployment
- **Deployment** – catalog-approved entities flow into the Gateway for enforcement
- **States** – Draft → Review → Approved → Deployed
- **Versioning** – entity-level versioning with support for multiple versions, coexistence, and deprecation

Federation & Interoperability

- Sync with **external registries** (Docker MCP Catalog, open MCP Registry, partner catalogs)
- Support **internal/private catalogs** while maintaining federation with external sources
- Provide **exportable configs and APIs** to integrate with customer-managed gateways or catalogs

Observability & Audit

- Track lineage of all catalog entities and their changes
- Provide **audit logs** for compliance and security reviews
- Surface usage data for administrators and platform engineers

---

Non-Goals (Initial Phase)

- Not duplicating Registry functionality (servers remain registry-owned).
- Not serving as runtime enforcement (gateway responsibility).
- Not tied to a single vendor product implementation; design must remain **agnostic and interoperable**.

Example Use Cases

1. Security team revokes a compromised tool across workloads instantly without waiting for server updates.
2. A finance workload can **read** Jira tickets but not **update** them.
3. A multi-purpose tool is simplified into distinct variants (e.g., get_issue, get_PR).
4. A new server version is introduced, workloads are migrated selectively while old version is deprecated.
5. Catalog exports governance rules to a customer's existing gateway for enforcement.

Integration Points

- **Registry** – Provides authoritative server metadata; Catalog enriches it with entities, mappings, and governance metadata.
- **Gateway** – Consumes catalog-approved entities and rules for runtime enforcement.
- **External Registries & Catalogs** – Sync with Docker MCP Catalog, open MCP Registry, Smithery, Microsoft Copilot Catalog, Databricks Unity Catalog, etc.
- **AI Hub (Future)** – Broader catalog integration unifying MCP entities with models, datasets, prompts, and agents.

Future Expansion

- Extend to full **AI Assets Catalog** (models, datasets, prompts, agents).
- **Agent Catalog** – manage agents as first-class entities and integrate with a future Agent Gateway.
- **Policy attestation & SBOMs** (e.g., Toolhive, JFrog AI Catalog practices).
- **Federation across multiple catalogs/registries** (catalog of catalogs).
- **Multi-tenancy support** for enterprises with multiple teams or business units.
- Enhanced observability and policy integration with Kuadrant, OPA, or similar policy engines.

Areas for Further Research

- **Versioning Model** – per-entity vs. catalog-wide vs. hybrid (industry shows both patterns).
- **Gateway Integration** – define the exact interface/artifacts linking catalog metadata to runtime enforcement.
- **Agent Expansion** – determine how catalog extends to cover agents and integrate with a future Agent Gateway.
- **Scalability Benchmarks** – validate performance at enterprise scale (10k+ assets).
- **UI/UX** – define intuitive ways to expose catalog information in RHOAI without overwhelming users.

Catalog Terminology

Catalog Terminology and Suggested Taxonomy

The term *catalog* is widely used across technology domains, but it can mean different things depending on what it indexes or enables. In MCP specifically, "catalog" can refer to curated directories, open marketplaces, or vendor-specific hubs. These categories often overlap, which creates ambiguity. To help align teams, the following baseline taxonomy and terminology are suggested.

Layer	Canonical Term	Example Name	Managed By	Visibility
L5 – External	*Marketplace / Community Directory*	Smithery, GitHub	External Providers	Public
L4 – Public Ecosystem	*Reference Catalog*	Red Hat Public Catalog	Red Hat	Public Web
L3 – Enterprise Federation	*Federated Catalog*	Additional Enterprise Catalogs	Client Teams	Enterprise-specific
L2 – Private Execution	*Execution Catalog*	RHOAI Global (Cluster) Catalog	Platform Engineering	Enterprise-level
L1 – Developer / Runtime	*Developer Platform Catalog*	RHOAI AI Asset Endpoints	Platform Engineering	Namespace-level

Current Usage Patterns

Different domains already use the term catalog with distinct connotations:

- **Marketplace Catalogs**: Provide a browsable collection of third-party integrations, often with install or purchase flows (e.g., Smithery, MCP.so).
- **Containerized Catalogs**: Focus on curated, packaged entries that can be run locally (e.g., Docker MCP Catalog).
- **Community Catalogs/Directories**: Open, community-driven lists or directories, often broad but lightly governed (e.g., MCPServers.org, Pulse MCP).
- **Developer Platform Catalogs**: Embedded into a dev ecosystem or product to make discovery native (e.g., GitHub MCP Registry, Postman getmcp).

- **Public Reference Catalogs**: Static or semi-curated sites meant to index MCP servers for awareness (e.g., mcpcatalog.io, MCP Market).

Suggested Taxonomy for Our Use

To reduce ambiguity, we should use modifiers or more precise terms instead of just "catalog."

1. **Marketplace Catalog**

- **Contains:** Broad set of MCP servers/tools with metadata, categories, and install flows.
- **Use Case:** When the catalog functions as a marketplace for discovery and adoption.
- **Examples:** Smithery, MCP.so.

2. **Execution Catalog**

- **Contains:** Curated entries packaged for execution (e.g., containers, binaries).
- **Use Case:** When trust, versioning, and reproducibility matter, and the catalog bridges directly into runtime.
- **Examples:** Docker MCP Catalog.

3. **Community Directory**

- **Contains:** Community-maintained lists, usually with categories and search.
- **Use Case:** When breadth of coverage and grassroots contributions are the primary goal.
- **Examples:** MCPServers.org, Pulse MCP.

4. **Developer Platform Catalog**

- **Contains:** Entries surfaced natively in a development ecosystem, tied to repo or workspace flows.
- **Use Case:** When integrating MCP discovery into developer workflows.
- **Examples:** GitHub MCP Registry, Postman getmcp.

5. **Reference Catalog**

- **Contains:** Public-facing indexes of MCP servers and implementations with categories.
- **Use Case:** When the catalog exists primarily for reference, awareness, or lightweight browsing.
- **Examples:** mcpcatalog.io, MCP Market.

Discovery vs. Execution Catalogs

Another useful distinction is whether a catalog **only points to assets** or also enables **direct execution**:

- **Discovery Catalog:** Holds metadata and pointers, enabling search and exploration but not runtime. (e.g., MCPServers.org, mcpcatalog.io).
- **Execution Catalog:** Supports direct execution or install, bridging discovery into runtime (e.g., Docker MCP Catalog, Smithery).

Recommendation for Internal Consistency

When naming or designing systems, suffix the catalog type to make intent clear. Examples:

- **SmitheryMarketplaceCatalog** (broad discovery + install)
- **DockerExecutionCatalog** (verified container-based execution)
- **CommunityDirectoryCatalog** (community-contributed breadth)
- **PlatformCatalog** (embedded in GitHub/Postman ecosystems)
- **ReferenceCatalog** (lightweight public index)

If a catalog spans multiple categories, use **Federated Catalog** to emphasize its cross-domain or multi-source nature.

Summary

By applying consistent modifiers and distinguishing between **discovery vs. execution roles**, we reduce ambiguity in how we describe MCP catalogs. This taxonomy creates a common baseline for discussing, evaluating, and designing catalogs across open marketplaces, containerized execution hubs, community directories, developer platforms, and public references.

📋 Catalog Data

Potential Data the Catalog Would Hold

Last Updated: 2025-10-18

A structured breakdown of the **data an MCP Catalog** could hold, organized by **core entities**, **curation layers**, and **cross-registry metadata**. The Catalog is designed to **aggregate and unify multiple MCP registries** (internal, partner, external) to enable **enterprise-wide discovery, search, and consumption** of MCP assets; servers, agents, models, guardrails, datasets, and more.

---

1. Core Entity: Catalog Entry

Represents any **discoverable AI asset** surfaced from one or more registries.

Field	Description
catalog_id	Unique identifier within the catalog
asset_type	Enum: MCP Server, Agent, Model, Guardrail, Dataset, Evaluation, Policy
title	Human-readable name of the asset
summary	Brief, marketing-friendly description
source_registry_ref	One or more registry sources that contain this asset
registry_entry_ids	IDs of the same item across multiple registries
visibility_scope	Enum: Global, Project, Namespace, Private

---

2. Aggregated Identity & Provenance

Field	Description
origin_type	Internal / Partner / Community / External

Field	Description
maintainer_ref	Contact or team maintaining the source entry
organization_name	Company, team, or contributor org
provenance_chain	List of registry sources and ingestion timestamps
asset_version	Aggregated semantic version from source registries
upstream_url	Link to original source (GitHub, OCI, or external catalog)

---

3. Metadata & Taxonomy

Field	Description
tags	Curated topic tags (analytics, nlp, ops, governance, etc.)
categories	Hierarchical classification (e.g., "Compute → Agentic → Routing")
domain_area	Business or technical domain (healthcare, finance, ops)
capability_summary	Human-readable description of what this asset enables
capability_keywords	Synonyms and search-optimized keywords
supported_protocols	e.g., MCP v1.18, OpenAI Responses API, REST, gRPC
supported_models	Models or backends the asset interoperates with
compatibility_matrix_ref	Link to version compatibility data

---

4. Multi-Registry Federation Data

Field	Description
registry_sources	List of registries from which this item was imported
deduplication_hash	Hash used to merge duplicates from multiple registries
merge_conflict_log	Record of any conflicting metadata fields
federation_status	Enum: Unified, Partial, Conflicted, Deprecated
last_synced_at	Timestamp of most recent sync from any registry
sync_policy	Enum: Auto, Manual, Scheduled, Disabled

---

5. Curation & Editorial Layer

Field	Description
curation_status	Enum: Suggested, Reviewed, Featured, Deprecated
curated_by	Red Hat, Partner, or Community curator
review_notes	Human editorial comments
rating_score	Average community or internal rating (0–5)
usage_popularity	Count or percentile of usage across environments
highlight_flags	e.g., “Editor’s Pick”, “Recently Certified”, “Trending”
feature_image_ref	Link to image/thumbnail for catalog UI display

---

6. Lifecycle Awareness (Read-Only from Registry)

Field	Description
registry_lifecycle_state	Snapshot of state from the registry (Certified, Quarantined, etc.)
validation_status	Pass/Fail summary from the registry
last_certified_at	Timestamp from registry
certification_ref	Link back to registry certification report
deprecation_notice	Text note if registry marks as deprecated

---

7. Consumption & Integration Data

Field	Description
connection_instructions	How to connect or deploy this asset (e.g., Helm, MCP connect command)
client_compatibility	Supported SDKs or interfaces (Python, JS, Go)
endpoint_ref	Live endpoint link (if MCP server or model endpoint)
deployment_template_ref	YAML/ConfigMap or Operator metadata for deployment
example_usage_refs	Code snippets, demos, or notebooks
related_assets	Models, agents, datasets, or policies commonly used together

---

8. Governance & Access Control

Field	Description
rbac_policy_ref	Policy defining who can view or consume the entry
compliance_tags	e.g., FedRAMP, GDPR, HIPAA
license_type	Open-source, Partner, Proprietary, Evaluation
terms_of_use_ref	URL or file path to license terms
audit_log_ref	History of view/download events
approval_status	Enum: Pending, Approved, Rejected

---

9. Observability & Analytics

Field	Description
views_count	Total number of views in the catalog
downloads_count	Total deployments or fetches
average_rating	Weighted score of feedback
performance_metrics_ref	Links to evaluation results or monitoring dashboards
usage_trends	Historical graph of adoption and deprecation
popularity_rank	Calculated rank among similar assets

---

10. Cross-Asset Relationships

Field	Description
related_mcp_servers	IDs of MCP servers offering related tools

Field	Description
related_agents	AI agents that consume or depend on this asset
related_models	Associated model endpoints or families
related_guardrails	Policy or safety layers tied to this asset
related_datasets	Data sources connected to the asset
related_evaluations	Quality or benchmarking results
related_gateways	MCP Gateways exposing this asset virtually

---

11. Tool & Capability Data (for MCP-aware Assets)

Applies when the catalog entry represents or references an **MCP Server** (or virtual MCP via a Gateway). These records are synchronized from the Registry and enriched for discovery, ranking, and recommendation.

Field	Description
tool_count	Total number of exposed tools/functions
tool_summary	Overview of toolsets offered (e.g., data-querying, summarization, vector-search)
tools	Array of tool descriptors
tool_ref_schema	Schema version describing tool metadata (e.g., MCP Tool Schema v1.1)
tool_capability_tags	Functional groupings (analytics, retrieval, governance, file-ops, etc.)
featured_tools	Curated subset highlighted in catalog UI
tool_visibility	Enum: Public, Restricted, Internal

Field	Description
tool_discovery_score	Popularity or recommendation weight for ranking
tool_eval_ref	Link to evaluation or safety report per tool
tool_example_ref	Code snippet or usage example
tool_dependency_map	Graph of tool dependencies and relations
tool_last_updated	Timestamp of last synchronization
tool_origin_registry_ref	Registry source reference for tool metadata

Example JSON

```JSON
{
  "catalog_id": "mcp-server-rhoai-playground",
  "asset_type": "MCP Server",
  "name": "RHOAI Playground MCP",
  "tool_count": 12,
  "tools": [
    {
      "tool_id": "text-summarize",
      "name": "Text Summarization",
      "description": "Summarizes text using configured LLM",
      "input_schema": {"type": "string"},
      "output_schema": {"type": "string"},
      "category": "nlp",
      "capability_tags": ["summarization", "language"],
      "example": "summarize('Explain MCP lifecycle in 50 words')",
      "last_updated": "2025-10-18T11:22:00Z"
    },
    {
      "tool_id": "vector-search",
      "name": "Vector Search",
      "description": "Retrieves context from vector database",
      "input_schema": {"type": "embedding"},
      "output_schema": {"type": "document"},
      "category": "retrieval",
      "capability_tags": ["vector", "retrieval", "semantic"],
      "example": "vector_search('What is MCP Gateway?')",
      "last_updated": "2025-10-18T11:22:00Z"
```

```
    }
  ],
  "tool_ref_schema": "MCP Tool Schema v1.1",
  "tool_visibility": "Public"
}
```

---

12. Extended Metadata (for MCP Catalog 2.x and Beyond)

Field	Description
ai_governance_score	Weighted evaluation combining compliance, quality, and popularity
semantic_vector_ref	Embedding vector for semantic search and retrieval
knowledge_graph_ref	Graph node ID for contextual relationships
recommendation_weight	Dynamic ranking value
annotations	Arbitrary metadata extensions
localization_metadata	Multi-language descriptions and display names

---

Summary of Key Differences from the Registry

Concept	Registry	Catalog
Purpose	Lifecycle, validation, certification	Discovery, aggregation, consumption
Scope	Single domain	Multi-registry federation
Data Ownership	Source-of-truth	Aggregated + enriched view
Governance Level	Policy enforcement	RBAC + visibility control

Concept	Registry	Catalog
Primary User	Platform administrators	End users, developers, consumers
Tool Data	Full, authoritative	Mirrored + searchable
Data Flow	Registry → Catalog	Catalog ← multiple registries

Catalog Links

MCP Catalog - Functional Requirements (Josh Salomon)

MCP Catalog - functional requirements

🤝 Registry integration Requirements

MCP Catalog – Registry Integration Requirements

Last Updated: `2025-10-01`

To ensure the MCP Catalog functions seamlessly with the Anthropic MCP Registry Standard, the following requirements must be met:

Registry API Consumption

- The Catalog must integrate with the MCP Registry API for discovery, filtering, and retrieval of MCP server metadata.
- Queries must support filtering by tool type, resource type, version, authentication methods, and trust level.
- Results must be paginated and sortable to handle large registries efficiently.

Metadata Handling

- The Catalog must ingest the full metadata schema defined in the registry, including:
    - Server identifiers (canonical URL, name, ID)
    - Supported transports and endpoints
    - Tool and resource definitions with argument/input/output schemas
    - Authorization information (e.g., OAuth endpoints, supported scopes)
    - Versioning and deprecation metadata
    - Signing and certificate data
    - Classification and tagging for search and filtering
- Metadata must be validated and normalized before surfacing in the Catalog.

Trust and Verification

- The Catalog must display the trust tier of each server (e.g., certified, community, unverified) as provided by the registry.

- Only verified or approved entries should be surfaced in enterprise contexts according to policy.
- Revocation or deprecation status from the registry must be enforced in the Catalog and reflected immediately to users.

Versioning and Deprecation

- The Catalog must present all supported versions of a server and clearly mark deprecated versions.
- Users must be able to pin to specific versions while being notified of newer recommended releases.
- Deprecated or retired entries must be hidden or flagged according to policy settings.

Authorization and Access Control

- The Catalog must respect visibility rules defined in the registry, ensuring that private or internal entries are not surfaced to unauthorized users.
- Registry-level scopes, RBAC, and approval workflows must flow through into Catalog views and search.

Health and Status Integration

- Health or monitoring data provided by the registry must be displayed in the Catalog to inform selection (e.g., uptime, latency, availability).
- Servers flagged as unhealthy must be deprioritized or removed from Catalog search results based on configuration.

Policy and Governance Alignment

- The Catalog must enforce enterprise governance policies by only surfacing entries that meet compliance standards.
- Scoping must be supported so that different business units or tenants see only the MCP servers approved for their use.

Federation and Multi-Registry Support

- The Catalog must support integration with multiple MCP registries (public and private).
- Federation and synchronization between registries must be supported, ensuring the Catalog can present a unified view across sources.
- Conflict resolution rules must be applied where the same server exists in multiple registries.

Search and Discovery

- The Catalog must expose advanced search across all registry metadata fields.
- Results must be filterable by capabilities, version, trust tier, health, and governance scope.
- Search must remain performant even as the number of registered MCP servers scales.

Security and Auditability

- The Catalog must consume and enforce registry-level security signals, including revocations, vulnerability flags, and signature/key validation.
- Catalog actions (e.g., surfacing of new servers, removal due to revocation) must be logged for audit purposes.

👹 Competitor Analysis

MCP Catalogs - Competitor Analysis (catalog-focused)

Snapshot of the main catalogs

- **Smithery** — Large open marketplace/catalog with rich search, categories, and install flows (CLI/SDK; hosted or local). (Smithery)
- **Docker MCP Catalog (+ Toolkit)** — Curated catalog integrated with Docker Hub/Desktop; entries are **verified, versioned** container images with an immediate "discover → run locally" path. (Docker Documentation)
- **GitHub MCP Registry** — Catalog-like discovery hub inside the GitHub ecosystem; launch post positions it as a central place to **find and explore MCP servers**. (The GitHub Blog)
- **MCPServers.org ("Awesome MCP Servers")** — Community directory with categories, search, and a "Remote MCP Servers" list. (Awesome MCP Servers)
- **Postman "getmcp"** — Postman workspace that **lists and lets you try** MCP servers via collections; leverages Postman's verified publishers. (Postman)
- **MCP.so** — Community-run directory/marketplace for MCP servers; open submissions and browsing. (MCP.so)
- **Pulse MCP** — Daily-updated directory reporting live **server counts** with search/browse. (PulseMCP)
- **MCP Catalog (mcpcatalog.io)** — Public category-browsable listing site of MCP servers/implementations. (MCP Catalog)
- **MCP Market (mcpmarket.com)** — Third-party directory with searchable listings and curated collections. (MCP Market)

Comparison of catalog capabilities

Capability	Smithery	Docker MCP Catalog	GitHub MCP Registry	MCPServers.org	Postman getmcp	MCP.so	Pulse MCP	mcpcatalog.io	MCP Market
Role	Marketplace/catalog (open)	Catalog + runtime (container-centric)	Catalog-style discovery inside GitHub	Community directory	Curated workspace(s) to list/try	Directory/marketplace	Media directory w/ live counts	Public catalog site	Directory
Discovery UX	Rich search, categories	Catalog in Hub/Desktop	Central "home base" for discovering MCP servers	Categories + search + "Remote" view	Browse collections; try now	Categories /tags	Search + trend counts	Category browse	Search + featured
Install / "Run" path	CLI/SDK; hosted or local	One-click/local via Docker Desktop/CLI	Delegated to client flows	–	"Try" via Postman collections	Link-outs (varies)	–	–	Link-outs
Trust signals	Publisher identity; hosted vs local	**Verified, versioned** images; container isolation	Repo provenance + GitHub identity	Community moderation	Postman verified publishers	Site moderation (varies)	Shows counts/updates	Site-governed	Thin/varies

Capability	Smithery	Docker MCP Catalog	GitHub MCP Registry	MCPServers.org	Postman getmcp	MCP.so	Pulse MCP	mcpcatalog.io	MCP Market
Metadata depth	Descriptions, categories, install info	Versions, packaging, partner tags	Listing & exploration context	Basic directory fields	API steps via collections	Varies; broad coverage	Directory fields + counts	Basic categories	Basic + curated sets
Programmatic hooks	CLI/SDK	Docker CLI & Hub APIs	GitHub APIs	–	Postman APIs	–	–	–	–
Scale / breadth (indicative)	Positions as "largest open marketplace"	"Verified/curated" set	New hub; backed by GH ecosystem	Hundreds and growing	Scoped to workspace curation	Large open directory	6k+ servers (live count)	Unstated	Unstated
Best fit	Breadth + quick install	Governed, repeatable local runs	Dev-centric discovery	Broad community sweeps	POC/demo "try-it"	Long-tail sweep	Market sizing & trends	Simple public browse	Supplemental browse

Sources: Smithery docs/site; Docker MCP Catalog/Toolkit docs; GitHub launch post; MCPServers.org (incl. Remote Servers); Postman getmcp workspace; MCP.so directory; Pulse MCP live directory; mcpcatalog.io site; MCP Market site. (Smithery)

Practical guidance (drop-in bullets)

- **Breadth discovery & fast onboarding:** Start with **Smithery** for wide coverage and quick "catalog → install" flows. (Smithery)

- **Governed execution path:** Use **Docker MCP Catalog** when you want curated entries with verification/versioning and a direct, containerized run path. (Docker Documentation)
- **Developer-native findability:** Reference the **GitHub MCP Registry** to meet developers where they already are and to leverage repo provenance. (The GitHub Blog)
- **Community sweeps & examples:** Keep **MCPServers.org** and **MCP.so** in your research loop for long-tail and category scans; treat verification as light. (Awesome MCP Servers)
- **Hands-on evaluations:** Use **Postman getmcp** for "try-it-now" evaluations and shareable testing flows. (Postman)
- **Market pulse:** Cite **Pulse MCP** when you need live ecosystem size snapshots and trend signals. (PulseMCP)

AI Asset Registry

📘 Unified AI Asset Registry & Catalog

Based on Red Hat AI Team Discussions – June 2, 2025

🎯 Vision & Purpose

The overarching goal is to establish a **unified AI asset registry** system that supports *MCP servers, agents, models, and additional components* across the Red Hat AI portfolio (OpenShift AI, RHEL AI, Inference Server, etc.). This system is being designed to support:

- **Discover** (browsing and filtering assets)
- **Governance** (security, RBAC, policy enforcement)
- **Deployment** (across single or multi-cluster environments)
- **Observability** (tracking performance, lifecycle, usage)

The **Registry** will act as the backend system of record, while the **Catalog** provides a curated frontend for approved and consumable assets.

📦 AI Asset Types and Lifecycle

🔧 Registry as a System of Record

A unified registry backend is being pursued, with the following assets considered in scope:

Asset Type	Status / Importance
Models	MVP-critical, includes open-source and vendor models
MCP Servers	Critical for agent orchestration and tool routing
Agents	Post-MVP
Evaluations	Post-MVP

Asset Type	Status / Importance
Guardrails	Post-MVP
Knowledge Sources	Post-MVP

Assets will go through **ingestion**, **security validation**, **policy compliance**, and then be exposed in the **Catalog** based on RBAC and filtering logic.

🛠️ Key Functional Requirements

🛡️ Security & Governance

- **Registry is curated**—assets must be approved before visible in the catalog
- MCP and agent ingestion flows must include **security scanning and signing**
- Governance mechanisms (e.g., policy-based model allow lists) are critical for enterprise compliance

🔌 Deployment Logic

- Deployment needs to support both **single-cluster** and **multi-cluster** environments
- Expectation that not all assets will be OCI-compliant (e.g., non-container artifacts)
- Important to not over-index on containers if alternate formats are more appropriate (e.g., raw files)

🧱 Technical Architecture & Compatibility

📚 Multi-backend Registry

- Although the experience will present as a single registry, **each asset type may have a unique backend and ingestion pattern**
- A potential **registry proxy** layer is being considered to translate across these asset types and expose a unified interface post-MVP (Q1 2026 goal)

📡 API & Automation

- APIs are critical for non-UI use cases:
  - Automation (e.g., DevOps pipelines)
  - Agent-to-agent interactions
  - Headless discovery and evaluation
- Reference implementations like [Anthropic's MCP Registry]() are being examined for MCP-related API patterns

🌐 Disconnected Mode

- **Disconnected environments are a known constraint**, especially for enterprise OpenShift clients
- Long-term strategy will be required for syncing/downloading selected assets (e.g., via Quay or equivalent)

🧩 UX Insights from Research (April 2025)

User Mental Models (per research by Jenn, Marc, and team)

Term	Persona	Common User Interpretation
Available Assets	AIE	Approved assets I can explore and use
Model Registry	PE	Internal versioning and tracking system
Model Catalog	PE	Search, filter, browse process (action-oriented) Possibly external source, possibly involving cost/licensing.

Notable UX Preferences

- Users often do **not distinguish registry vs. catalog** clearly
- App engineers/developers treat the Available Assets - MCP page as their **only interface**, unaware of the registry's existence. Keeping them Kube-blind
- Preferred views differ:
  - **Card view** favored for browsing available assets
  - **List/Table view** favored for personal/team/internal assets
  - Expects quick search/filter

🧭 Strategic Considerations

💡 Design Philosophy

- Avoid trying to solve cloud-native and RHEL-native use cases simultaneously in MVP
- Focus first on OpenShift AI as primary SKU, then **expand to Inference Server, RHEL AI, Developer Hub**
- MCP registry/catalog experience could be surfaced through **Developer Hub plugins** in the future

🪙 Value to Enterprise

- Emphasis on enabling **enterprise policy controls**
- Providing visibility into what's running, where, and why
- Reducing "Wild West" deployment of models and agents via structured governance

---

📌 Recommendations for MVP

MVP Scope (Post-KubeCon)

- Unified registry with support for:
  - **Models**
  - **MCP Servers**
- Shared catalog UI with clear segmentation and RBAC support
- API-first ingestion and access
- Manual registration plus metadata ingestion from known sources (e.g., HuggingFace, Anthropic)
- Secure deployment from Registry → Catalog → Inference

Post-MVP Explorations

- Agents and Evaluations
- Registry backend harmonization (proxy/adapter layer)
- Disconnected sync workflows
- RBAC federation and user scoping

- Developer Hub integration

📇 Registry Terminology and Suggested Taxonomy

Registry Terminology and Suggested Taxonomy

The term *registry* is widely used across technology domains, but it can mean very different things depending on what it contains. In some cases, a registry holds human-focused information about servers. In others, it manages maintained images or even exposes active server processes that can be connected to. These categories are not mutually exclusive, which creates ambiguity. To help our teams align, the following baseline taxonomy and terminology are suggested.

Current Usage Patterns

Different domains already use the term *registry* with specific connotations:

- **Container / Image Registries** Hold container images, artifacts, and tags. Provide storage, distribution, and discovery of binaries.

- **Metadata Registries** Contain schemas, data definitions, and descriptions. Provide information *about* data rather than the data itself.

- **Service / API Registries** Maintain endpoints and metadata about services or processes. Used for service discovery and connection.

- **MCP Registries** Provide discovery of MCP servers, describing what servers exist and how to connect to them. Typically function as a "source of truth" for discovery, not hosting the servers themselves.

Suggested Taxonomy for Our Use

To remove ambiguity, we should use modifiers or more precise terms instead of just *registry*.

1. Service (or Endpoint) Registry

- **Contains:** URLs or endpoints and associated metadata (capabilities, versions, authentication).
- **Use Case:** When the registry is primarily about connecting to live services or processes.

2. Artifact (or Image) Registry

- **Contains:** Artifacts, images, or binaries such as container images or compiled modules.
- **Use Case:** When the registry is primarily about storing and distributing software artifacts.

3. Data Registry

- **Contains:** Data sets, files, or pointers to structured data sources.
- **Use Case:** When the registry is about exposing or organizing data.

4. Metadata Registry

- **Contains:** Schemas, attributes, definitions, or ontologies.
- **Use Case:** When the registry provides descriptions of data or services rather than the assets themselves.

5. Catalog / Directory

- **Contains:** A broad index or pointer layer that may reference multiple types (services, data, artifacts).
- **Use Case:** When one registry spans multiple categories or acts as a federated index.

Discovery vs. Storage Registries

Another useful distinction is between *discovery* and *storage* functions:

- **Discovery Registry**: Holds only metadata or pointers that allow clients to find and connect to assets. Does not host the assets themselves.
- **Storage Registry**: Hosts the actual assets (images, binaries, or datasets).

This distinction can apply across all categories (e.g., a "Discovery Service Registry" vs. a "Storage Artifact Registry").

Recommendation for Internal Consistency

When naming or designing systems, suffix the type of registry to make intent clear. Examples:

- UserServiceRegistry (endpoints for user services)
- ImageArtifactRegistry (binary artifacts)
- ClinicalDataRegistry (datasets)
- SchemaMetadataRegistry (metadata definitions)

If a registry spans multiple types, call it a **Catalog** or **Directory** to emphasize its cross-domain nature.

Summary

By using consistent modifiers and making the distinction between discovery vs. storage roles, we can reduce ambiguity and help our teams communicate more effectively. This taxonomy provides a common baseline for discussing, designing, and maintaining registries across services, data, and artifacts.

Responses to Ramesh's Questions

1. What does an MCP Server represent?

An **MCP Server** is the runtime endpoint that implements the Model Context Protocol — effectively a *service interface* exposing one or more tools, data connections, or actions that an agent can call.

It is *not* a model or inference endpoint, though it's conceptually parallel. If you think of an inference endpoint as a hosted LLM exposing a /predict or /chat API, an MCP server exposes **contextual tools or services** over standardized transports (http, ws, stdio, etc.).

An MCP Server:

- Runs somewhere accessible (on-prem, in-cluster, or public).
- Responds to standardized requests per MCP spec.
- May host multiple tools or sub-services.
- Can be versioned, replaced, or retired — just like model endpoints.

---

2. What is the equivalent of the "Model Registry"?

The **MCP Registry** is the analog of the Model Registry. It stores and manages **MCP Server metadata**, *not* the runtime instances themselves. Each record in the registry corresponds to a *server definition*, containing fields like:

- id, name, version, description
- connections (URIs, transport types, configs)
- license, authors, categories, tags
- Lifecycle metadata (createdAt, updatedAt)
- Certification and security status (x-certificationStatus, x-securityScan, x-maturityLevel)

This metadata allows for:

- Discovery of available MCP Servers
- Validation, quarantine, or certification workflows
- Governance and version tracking across environments

When a runtime MCP server comes online, it can register itself or be registered manually, but the registry is always the **system of record** for its metadata.

---

3. Is it a registry of runtime servers or metadata alone?

It is primarily **a registry of MCP Server metadata**. Runtime MCP processes are *referenced* through connection URIs and health/status data, but they aren't *stored* within the registry. You can think of it as:

Layer	Description
MCP Server (runtime)	Live process exposing MCP endpoints
MCP Registry	Metadata and governance layer referencing servers
MCP Catalog	Public/federated layer aggregating many registries

---

4. What does an MCP Catalog mean?

The **MCP Catalog** federates one or more registries — both internal and external. It's the read-only discovery surface that aggregates servers from sources like:

- GitHub MCP Registry
- Anthropic / Simathry public registries
- ToolHive ecosystem registry
- Red Hat internal "blessed" partner registries
- Customer-defined or on-prem registries

The Catalog is *open and federated*, while the Registry is *controlled and governed*. Access flow looks like this:

```
None
MCP Catalog (Read-Only)
            ↓
MCP Registry (Curated / Certified)
            ↓
Llama Stack / AI Studio Consumption
```

---

5. Where do validation, certification, and governance belong?

Those controls belong primarily in the **Registry**, not the Catalog.

- The **Catalog** should remain open for discovery.
- The **Registry** is where MCPs are imported, quarantined, scanned, validated, certified, and versioned.
- Once approved, they can be promoted to enterprise-wide visibility or surfaced back to the Catalog under a "certified" tag.

This mirrors the model lifecycle pattern (Catalog → Quarantine → Certified → Published).

---

6. Nomenclature and ToolHive alignment

ToolHive (Stacklok) is already using terms that align well:

- **Registry** → structured store for MCP or model metadata.
- **Catalog** → federated, read-only discovery layer.
- **Lifecycle** → pipeline for scanning, validation, and promotion.

Our schema (MCP Server Metadata) maps cleanly to that convention. We should maintain that alignment so that future interoperability between Red Hat's registry and Stacklok/ToolHive's ingestion pipeline is seamless.

✅ Summary

Concept	Purpose	Analogy (Model World)	Governance
MCP Server	Live runtime exposing MCP tools and APIs	Model Inference Endpoint	Operational
MCP Registry	Stores metadata, manages lifecycle, certifies	Model Registry	Controlled (RBAC, validation)
MCP Catalog	Federated discovery of external sources	Model Catalog	Read-only
Certification Pipeline	Validates & promotes MCPs from Catalog to Registry	Model certification	Policy-driven

---

Would you like me to format this as a formal **"MCP Registry & Catalog Alignment"** section for your document, with Red Hat-style headers and structure? It would be ready to drop into your taxonomy file.

📋 Registry Data

Potential Data the Registry Would Hold

Last Updated: 2025-10-18

A structured breakdown of the **data an MCP Registry** could hold, organized by **core entities**, **metadata categories**, and **governance fields**. This aligns with Red Hat's MCP lifecycle/governance model and the Anthropic MCP spec.

---

1. Core Entity: MCP Server Record

Represents a registered MCP server (or MCP-compliant endpoint).

Identity & Basic Metadata

Field	Description
server_id	Unique identifier (UUID or hash) for the MCP server
name	Human-readable name (e.g., "Starburst Query MCP", "RHOAI Playground MCP")
description	Short description of the server's purpose
owner	Account, team, or org that owns the MCP server
maintainer_contact	Email, RHID, or contact for lifecycle notifications
registry_origin	Whether internal, external, or third-party (e.g., Red Hat, Partner, Community)

---

2. Connection & Discovery Data

Field	Description
endpoint_url	Base URL where the MCP server is accessible

Field	Description
protocol_version	MCP protocol version supported (e.g., 1.18.0)
auth_type	Type of authentication (OAuth2, API key, RH-SSO, none)
token_policy_ref	Link to secrets/credentials policy or secret object
network_zone	Label for cluster/network boundary (public, internal, air-gapped)
reachable	Boolean or last-check result for availability
last_ping_time	Timestamp of last successful handshake
connection_health	Enum: Healthy / Warning / Unreachable

---

3. Capability & Schema Data

Field	Description
tools	List of tools exposed (names, parameters, schema)
resources	List of resources offered (files, models, agents, data endpoints)
models_supported	Associated model endpoints (if applicable)
vector_schema	Vector store schema (if applicable)
graph_schema	Graph schema (if applicable)
function_count	Number of callable tools/functions
capability_summary	Text summary of the server's domain and scope
capability_tags	Tags like analytics, data, llm, governance, policy

---

4. Lifecycle & Certification

Field	Description
lifecycle_state	Enum: Registered, Quarantined, Validated, Certified, Deprecated, Retired
onboarding_source	Method: Manual, Auto-discovered, External registry import
validation_report_ref	Link to scan/validation report
security_scan_status	Enum: Pass / Fail / Pending
vulnerability_score	CVSS or Red Hat internal risk score
certified_by	Red Hat or partner authority
certification_date	Timestamp of last certification
review_cycle	Periodic review cadence (e.g., 90 days)

---

5. Versioning & Provenance

Field	Description
version	MCP server version or commit hash
registry_version	Version of this entry in the registry
created_at	Creation timestamp
updated_at	Last update timestamp
provenance_ref	Git or OCI reference for provenance
change_log	Structured list of metadata or capability updates

---

6. Governance & Policy

Field	Description
rbac_policy_ref	Link to access control definition
data_sensitivity	Enum: Public, Internal, Confidential
compliance_tags	e.g., GDPR, HIPAA, FedRAMP
usage_policy_ref	Pointer to usage restrictions or SLAs
audit_log_ref	Log of registration and access events
license_type	Open-source, proprietary, partner-licensed
terms_of_use_ref	Link to license or ToS document

---

7. Relationships & Cross-Links

Field	Description
related_models	Model registry IDs this server uses or exposes
related_agents	Agents or applications consuming this MCP
related_catalog_entries	Corresponding entries in the AI Catalog
related_guardrails	Associated policy or moderation MCPs
cluster_ref	Cluster or namespace this MCP belongs to
gateway_virtualization_ref	Gateway context if served via a virtual MCP

---

8. Observability & Metrics

Field	Description
usage_stats	Total calls, average latency, error rates
uptime_percent	Rolling uptime metric
last_used	Timestamp of last consumption event
observability_ref	Link to Prometheus / Grafana metrics
performance_grade	Optional performance evaluation from evaluation MCP

---

9. Extended Metadata (for MCP Registry 2.x and beyond)

Field	Description
ai_evaluation_score	Quality or safety rating from evaluation pipeline
guardrail_status	Whether guardrails or policies are active
sandbox_ref	Temporary testing environment link
compatibility_matrix	Compatibility with MCP clients or SDK versions
annotations	Arbitrary key/value metadata for extensions

🔗 Registry Links

ToolHive Kubernetes Operator - ToolHive Kubernetes Operator - Project Assessment
Registry Operator ADR - MCP Registry Operator - ADR

🔌 Interfaces Exposed

Interfaces Exposed

Below is a quick explainer of each interface the MCP Registry exposes and **why it matters to users and operators**.

Capability	What It Is	Why It's Important
OpenAPI & GraphQL endpoints	*Machine-readable APIs* that let any tool query the catalog (search/filter), publish new server versions, promote them to "certified," or deprecate old ones. OpenAPI offers predictable REST calls; GraphQL gives front-ends a flexible, one-query-gets-exact-fields model.	**Ease of integration** – CI pipelines, custom UIs, or third-party portals can interact with the registry without custom adapters. • **Future-proofing** – GraphQL prevents "version explosion" as new metadata fields are added; consumers ask only for what they need. • **Automation-friendly** – teams can script bulk uploads, rollbacks, or metadata updates directly from their build systems.
Webhooks / GitOps feeds	The registry emits signed notifications whenever a server is added, updated, or flagged by a CVE scan. Feeds include the generated **SBOM** (software bill of materials) and a **CVE-diff** payload so downstream tools know exactly what changed.	**DevSecOps governance** – security teams receive real-time signals to trigger vulnerability management workflows. **GitOps sync** – clusters following GitOps pull-requests receive automated updates, keeping desired-state repositories in lock-step with certified assets. **Audit & compliance** – signed SBOMs satisfy supply-chain requirements (e.g., NIST SSDF, EU CRA).

MCP-Client Helper (CLI/SDK)	A small client library/CLI that resolves URIs like mcp://registry/github@1.2.0 into the correct image digest plus metadata. It handles auth, cache, and optional policy checks before an agent starts using the server.	**Developer ergonomics** – engineers reference a simple URI instead of hard-coding container digests or endpoints. **Version pinning & rollbacks** – the helper locks agents to a specific, immutable version and can auto-prompt upgrades when approved. **Policy enforcement** – client can refuse to pull uncertified or end-of-life servers, preventing rogue tool usage at runtime.

Bottom line: these interfaces make the MCP Registry *programmable, observable, and enforceable*—key traits for scaling AI toolchains across multiple teams and clusters without sacrificing security or developer velocity.

📦 Server Deployment

MCP Server Deployment & Registration Spec

📌 1. Purpose

This spec defines the standard for **deploying**, **registering**, and **certifying MCP Servers** in a Kubernetes-native environment. It aligns with the RHOAI – MCP Registry and Catalog system to ensure consistent behavior across Helm, Operators, and GitOps workflows.

It does **not mandate** a deployment mechanism but **standardizes expectations** for metadata, registration endpoints, and lifecycle transitions.

🧩 2. Components

Component	Description
MCP Server	A containerized server exposing natural language control for an application
Deployment Mechanism	Helm, Operator (via OLM/CSV), or GitOps manifest
MCP Registry	The system of record for registering, scanning, certifying, and indexing MCP Servers
Catalog	Read-only index of certified MCP Servers used at runtime by agents and users
Mirror Controller	Operator responsible for mirroring external registries and managing lifecycle CRDs

---

🧪 3. Specification

3.1 Deployment Mechanisms

- MCP Servers **MUST** be deployed using standard Kubernetes primitives:
  - Deployment or StatefulSet
  - Service for endpoint exposure
  - ConfigMap or Secret for configuration
- The deployment MAY be initiated via:
  - A Helm chart
  - An Operator (using a CSV)
  - A GitOps pipeline
  - A MirrorPolicy CRD (via Mirror Controller)
- The MCP Server container **MUST include** a metadata.json file at the root of the image or a known path (/etc/mcp/metadata.json) for registry extraction.

3.2 Metadata Requirements

On import or registration, the following **MUST** be provided either via API or extracted from the container image:

🧾 Required Fields:

Field	Description
id	Unique ID (slug) for the MCP server
name	Friendly name (e.g., "ArgoCD Control")
description	Full description of MCP Server
version	Version of the MCP server
license	SPDX license identifier (e.g., "Apache-2.0")
toolList	List of available tools
mcpSpecVersion	Version of the MCP spec followed (e.g., "1.0.0")
connections[]	Array of connection objects (see below)

Field	Description
createdAt	ISO timestamp
updatedAt	ISO timestamp

🔗 Connection Schema:

```JSON
{
  "type": "http",
  "uri": "http://mcp-argo.namespace.svc.cluster.local",
  "configSchema": {}
}
```

🧩 Optional Metadata Fields:

- authors[], maintainers[], tags[], categories[]
- dependencies[] (other MCP IDs or services)
- homepage, repository, docs
- description, logo, supportContact

🔒 Vendor Extensions (Populated by Registry):

Field	Description
x-securityScan	SBOM/CVE report (e.g., via Syft, Grype, Clair)
x-certificationStatus	"quarantined", "pendingReview", "certified", "deprecated"
x-maturityLevel	"experimental", "beta", "stable", "lts"
x-isOutdated	Boolean flag for outdated MCPs
x-recommendedVersion	Suggested upgrade version

3.3 Registration APIs

The following endpoints are available via the MCP Registry:

🚀 Import (Registry-Initiated)

```
None
POST /servers/import
```

- Accepts: OCI image URI or uploaded tarball
- Action: Extracts metadata.json, sets x-certificationStatus = "quarantined", triggers scan

🤖 Self-Registration (Optional)

```
None
POST /servers/self-register
```

- Called by MCP Server on startup
- Uploads metadata + endpoint info
- Triggers same quarantine + scan process as /import

3.4 Promotion & Certification (Manual)

🧪 Promote Server

```
None
POST /servers/{id}/promote
```

- Used by a human reviewer after reviewing scan results
- Sets x-certificationStatus = "certified"
- Triggers catalog index update and optional webhook

🛑 Deprecate Server

```
None
PUT /servers/{id}/deprecate
```

- Sets x-certificationStatus = "deprecated"
- Agents attempting to use this server may receive HTTP 410 or warning

---

3.5 Consumption & Upgrade Advisory

- MCP servers are consumed using:

```
None
mcp://registry/github@1.2.0
```

- Registry CLI or SDK will resolve using:

```
None
GET /resolve?name=github&version=1.2.0
```

- Registry may return:

```
JSON
{
  "currentVersion": "1.2.0",
  "recommendedVersion": "1.3.0",
  "metadata": {...}
}
```

- Agents MAY auto-upgrade if x-isOutdated = true.

3.6 Operator/Helm Integration

Operators or Helm charts **MAY** include annotations in metadata to indicate MCP server presence:

```
None
metadata:
  annotations:
    mcp.redhat.com/has-mcp-server: "true"
    mcp.redhat.com/server-image: "quay.io/org/mcp-argo:v1.3.2"
    mcp.redhat.com/registration-mode: "auto"
```

These annotations enable:

- Cataloging via OperatorHub
- Deployment-time registration automation
- Policy-based validations via Mirror Controllers

3.7 MirrorPolicy CRD (Optional Flow)

If mirroring external registries, use:

```
None
apiVersion: mcp.redhat.com/v1alpha1
kind: MirrorPolicy
metadata:
  name: github-mirror
spec:
  source:
    uri: "https://external-mcp-registry.com"
  sync:
    frequency: "24h"
    autoImport: true
    autoPromotePatchVersions: true
```

The Mirror Controller will:

1. Detect new MCP servers
2. Import them via /servers/import
3. Quarantine + scan
4. Auto-promote patch-level versions

✅ 4. Compliance Checklist

Item	Required	Notes
Kubernetes-native deployment	✅	Use Deployment + Service
metadata.json embedded in image	✅	Required for /import
Registry registration endpoint used	✅	/import or /self-register
Promotion endpoint honored	✅	/promote used by reviewer

Item	Required	Notes
x-certificationStatus lifecycle	✅	Transitions: quarantined → certified → deprecated
CLI URI support (mcp://)	✅	Must support /resolve
MirrorPolicy support (if applicable)	⬜	Optional, but recommended for large deployments
Helm/Operator annotations (optional)	⬜	Recommended for discovery via OperatorHub

---

🔄 5. Registry Interfaces Overview

Interface	Endpoint	Description
Import	POST /servers/import	Imports new MCP server metadata from container
Self-Register	POST /servers/self-register	Optional endpoint for MCP servers to register themselves
Search	GET /servers or GraphQL	Query MCPs by name, version, tag, etc.
Promote	POST /servers/{id}/promote	Approves and certifies a quarantined server
Deprecate	PUT /servers/{id}/deprecate	Flags server as deprecated
Resolve	GET /resolve?name=...&version=...	Retrieves image + metadata for use
Webhook	POST /webhooks/scan	Emits SBOM + CVE feed updates

Security Extensions

What are the x-securityScan, x-certificationStatus, and x-maturityLevel fields?

In the MCP Registry's metadata schema, everything that is *not* part of the official MCP spec is stored under custom "vendor-extension" keys (those that start with x-).

The three security-oriented extensions below record the **current trust posture** of each MCP server as it moves through the registry pipeline.

Field	Type / Example	Purpose in the Pipeline	How the Registry Uses It
x-securityScan	json\n\"x-securityScan\": {\n \"scanner\": \"Trivy 0.51.1\",\n \"timestamp\": \"2025-05-05T18:12:44Z\",\n \"summary\": {\n \"critical\": 0,\n \"high\": 2,\n \"medium\": 5,\n \"low\": 8\n },\n \"sbomDigest\": \"sha256:…\"\n}\n	Stores the latest automated scan results—CVEs by severity, the SBOM hash, and the tool version that produced them.	*Gatekeeper*: • Blocks promotion if critical/high CVEs exceed policy. • Surfaces severity counts in the UI. • Triggers webhook so SecOps can open tickets on new CVEs.
x-certificationStatus	"quarantined", "in_review", "certified", "deprecated", "rejected"	Tracks where the server sits in the approval workflow—from first import (quarantined) through human sign-off (certified) to eventual retirement (deprecated).	*Promotion logic*: • Catalog only shows entries where status = certified. • CLI refuses to pull rejected versions unless --force. • Operator can auto-decline any install that isn't certified.

x-maturityLevel	"experimental", "beta", "stable", "lts"	Communicates long-term support expectations to consumers, independent of certification. A stable, certified server is production-ready; an experimental one is "use at your own risk."	*UX & policy*: • Filters in the UI (e.g., "show only stable tools"). • Org-wide policy can forbid experimental on prod clusters. • Upgrade advisories recommend moving from beta→stable when available.

Why bundle these into the metadata?

1. **Single source of truth** – Every artifact carries its own trust signal; no external spreadsheet needed.
2. **Automation-ready** – Build pipelines, security scanners, and GitOps controllers can parse these JSON keys to make pass/fail decisions.
3. **Auditable history** – Because each version's metadata is immutable once published, you can prove *when* it was scanned, *by whom*, and *under what criteria* if auditors ask.

In short, these extensions turn a plain tool listing into a policy-enforced supply-chain record—ensuring that only *known-good, policy-compliant* MCP servers reach your AI agents.

MCP Gateway

🌉 MCP Gateway – Overview

Last Updated: 2025-10-01

OCP Networking MCP Documentation - 📄 MCP Gateway Project

Executive summary

OpenShift Networking, with the Agent/MCP team, IBM Research, and UX, is delivering an Envoy/Istio–based "MCP Gateway" that terminates MCP client traffic, discovers/aggregates backend MCP servers, exposes curated "virtual MCP servers," and enforces security, identity, and observability for Agentic AI on OpenShift. This sits alongside Lifecycle/Registry and AI Studio streams as the governance/runtime enforcement layer for MCP.

Industry context. A fast-forming competitive set (Solo's Agentgateway under the Linux Foundation; Kong AI Gateway; Gravitee Agent Mesh) validates the approach: gateways that federate tools, add fine-grained auth, and provide MCP/A2A observability. Our differentiation is deep OpenShift/Istio integration, Kuadrant policy, and an upstream-first path on Envoy rather than a bespoke proxy. (agentgateway)

Problem & value

Problem. Teams face tool sprawl across many MCP servers with inconsistent authn/z and limited inspection.

Value. The gateway composes selected tools into a single governed entry point; applies RBAC and per-tool controls; centralizes logging/metrics/audit. Outcome: simpler agent integration for developers and consistent enforcement/visibility for platform/security.

Scope (now vs later)

MVP / Developer Preview (short runway). Streamable HTTP MCP proxy (initialize/tools.list/tools.call + SSE), virtual MCP servers (tool federation, deterministic namespacing), identity-aware list filtering, allow/deny per-tool, external/in-cluster MCP

registration, and per-tool metrics/audit. Tracks the internal CONNLINK tasks tied to OCPSTRAT-2297.

Out of scope initially. Resumable sessions, capability types beyond Tools, sophisticated guardrails, token-based quotas, HA/scale, caching/LB, registry integration, A2A, stdio, gateway-starting backends.

Comparable solutions & prior art (concise)

Full Competitor Analysis - RHCL AI gateway competitor research

- **Solo Agentgateway (LF project):** AI-native gateway for MCP & A2A; virtual MCPs; session mgmt; fine-grained auth (Cedar). Solo built a dedicated proxy to move faster on stateful AI protocols. (agentgateway)
- **Kong AI Gateway:** OIDC/API keys, **token-based** AI rate limits, guardrail plugins, and MCP governance positioning; MCP server available for Konnect. (Kong Inc.)
- **Gravitee Agent Mesh:** Agent gateway + **MCP Tool Server** that wraps REST/OpenAPI as MCP tools; dev-portal discovery. (Gravitee)

Implication: we stay OpenShift-native on Envoy/Istio, integrate Kuadrant, and upstream key MCP features to Envoy for long-term supportability.

Architecture direction

Platform choice. MCP support as an Envoy extension running in our Istio-based Gateway API (leveraging RHCL/Kuadrant over time).

Extension strategy (updated).

- **DP**: implement JSON-RPC inspection/routing via **ext_proc** for speed of iteration.
- **GA target**: migrate to **Proxy-Wasm** for sandboxing/portability and RHCL delivery.
- **Upstream watch**: track Envoy's **MCP support proposal** and **stateful session** work to retire custom parsing/stickiness as native support lands. (GitHub)

Gateway responsibilities. Maintain front/back MCP sessions; curate tool belts into virtual servers with collision avoidance; parse & route tool calls; enforce per-tool policy; centralize observability; register both in-cluster & external MCP servers. (Research prototypes already cover initialize/tools.list/tools.call and session swap.)

Detailed requirements (acceptance)

- Streaming connections and SSE; list & call tools; compose/filter tool belts; prevent name collisions; identity-aware RBAC; audit/log/metrics; server registration & auth config. Mapped to CONNLINK-502..513.

Deployment considerations

All footprints (self-managed/managed; classic/HCP; multi/compact/SNO; connected/restricted; all arches). Istio/RHCL integration for delivery and policy. UI location TBD (Console plugin vs AI Studio).

Personas & use-cases

- **AI Engineer:** one virtual MCP exposing the right tools for a project.
- **Platform/SRE:** enforce who can use which tools; quotas; unified telemetry.
- **Security/Compliance:** choke-point for inspection and future guardrails.

Interfaces & integration

- **Agent→Gateway:** HTTP(S) MCP (initialize/tools.list/tools.call + streams).
- **Gateway→Servers:** sticky, persistent sessions; per-server auth.
- **Policy plane:** Kuadrant (ext-auth/rate limit); RBAC aligned to cluster identity.
- **AI Studio / Catalog / Registry:** register/discover virtual MCPs for agents.

Relationship to Lifecycle, Registry, Catalog (updated)

- **Lifecycle:** Toolhive (Stacklok) remains the lifecycle workstream for MCP servers. (Active operator/lifecycle alignment noted in meetings.)
- **Registry:** The **official MCP Registry** launched (Sept 8, 2025). Plan to **consume metadata** for external servers and **publish virtual MCPs** into an enterprise sub-registry (Toolhive-backed). (mcp blog)
- **Catalog:** Short-term "blessed ISV" list; converge with the official registry as it matures. Backstage prototype ongoing.

Security & policy integration (updated)

Authentication & identity. Adopt **MCP Authorization (OAuth 2.1)** for HTTP transport: validate bearer tokens at the gateway; support backend token exchange where needed; fall back to API keys for DP if necessary. (GitHub)

Fine-grained authorization.

- **DP:** allow/deny per tool mapped to identity; list-time filtering + call-time enforcement.
- **Next:** pluggable policy engines (Kuadrant ext-auth/OPA; later Cedar-style policies as seen in industry). (agentgateway)

Quotas & rate limiting (updated).

- **DP:** per-tool request counting.
- **Next:** introduce **token-based** limits and budgets (tokens/min) using Kuadrant descriptors—an approach consistent with industry plugins. (Kong Docs)

Auditing. Per-method audit logs (initialize/list/call) with identity, tool, outcome; integration with OpenShift logging/SIEM. (Semantic, tool-aware fields.)

Observability & guardrails (updated)

Telemetry. Per-tool counters, latency histograms, errors; OpenTelemetry spans per tools/call (trace across backends); session metrics.

Developer visibility. Roadmap a lightweight developer-portal/playground to list/debug tools for a virtual MCP—mirrors competitor UIs. (agentgateway)

Guardrails. Pluggable processors: prompt poisoning detection and PII/secret redaction first; ability to **mark guardrails** per tool/virtual server and enforce block/mask actions; extensible moderation webhooks. (Longer-term: semantic classifiers.)

Non-functional requirements

Security (RBAC by tool; auditable changes), reliability (sticky sessions, graceful degradation), maintainability (delivered via RHCL; minimal bespoke control plane), portability (track Envoy native MCP features as they land).

MVP → follow-ons (updated cut)

MVP / Dev Preview (minimum lovable gateway).

- Data plane: Envoy/Istio + **ext_proc** JSON-RPC inspection/routing; single-replica stickiness.
- Features: streaming proxy; **virtual MCP servers** (compose & namespace); **identity-aware filtering**; **basic authorization** (allow/deny per tool); **external/in-cluster MCP registration**; **audit + per-tool metrics**.
- Docs & samples: "bring your MCP", create a virtual MCP, call from agent.
- Known risks: OAuth/secret story needs Security engagement; unify backlog.

Release 1 — Hardening & Policy.

- Proxy-Wasm path for JSON-RPC; adopt Envoy **stateful session** work.
- OIDC validation; pluggable authZ via Kuadrant ext-auth; per-tool & per-caller rate limits; Otel traces; regex/PII redaction; mark/enforce guardrail policies. (GitHub)

Release 2 — Registry, Catalog, UX.

- Import MCP server metadata from **official MCP Registry**; publish **virtual MCPs** to an enterprise sub-registry; Console plugin for composition/policies; dev playground. (mcp blog)

Release 3 — Scale, Resilience, Cost.

- Multi-replica gateway with cross-replica stickiness; circuit breaking/outlier detection; token-based budgets; priority/fairness; cost indicators & alerts. (Interop tests against Solo/Kong/Gravitee for portability.) (agentgateway)

Risks & mitigations (tuned)

- **Timing vs bespoke AI gateways.** They already ship sessions, dev UIs, token quotas. Mitigation: prioritize streaming, per-tool RBAC, audit; upstream to Envoy (MCP proposal, sessions) and evaluate targeted adoption if upstream slips. (agentgateway)
- **Spec churn (OAuth 2.1, batching).** Maintain an MCP conformance test suite in CI and track June 2025 changes; validate OAuth resource metadata flows. (GitHub)
- **OAuth & secrets maturity.** Coordinate with Security PM; document DP auth limits; move to OIDC flows by R1.

Appendices

A — Acceptance criteria ↔ CONNLINK mapping

CONNLINK-502 (discovery), -503 (body parsing & tool extraction), -504 (session ID), -505 (SSE), -506 (observability), -508 (authn), -509 (authz), -510 (identity-filtered tools.list), -511 (rate limiting), -513 (per-server auth config).

B — Upstream tracking (Envoy)

- **MCP proposal** (body parsing/JSON-RPC awareness, method-based routing). (GitHub)
- **Stateful session** enhancements (header/cookie-based session affinity for MCP). (GitHub)

C — Meeting highlights shaping scope/roadmap

- **MVP focus**: authorization, virtual MCP, external MCP support; combined demo after MVP; OAuth gaps need Security engagement; unify backlog.
- **Dev Preview target & productization gates** (downstream, Conflux, docs; Summit-aligned 3.1/3.2).
- **Registering virtual MCPs** for discoverability & UX exploration underway.

Notes on what changed in this update

- Added **industry/prior-art** section and competitive positioning. (agentgateway)
- Clarified **extension strategy** (DP=ext_proc → GA=Wasm) and added **Envoy upstream watch** items. (GitHub)
- Elevated **OAuth 2.1** as baseline per **June 2025** updates. (GitHub)
- Introduced **token-based** quotas in roadmap (Kuadrant-backed), aligning with market practice. (Kong Docs)
- Aligned **Registry** plan with the **official MCP Registry preview** (Sept 8, 2025) and committed to registering **virtual MCPs** in an enterprise sub-registry. (mcp blog)

⏳ Status Updates

MCP Gateway Status Updates

10/20/2025 - Notes - MCP Gateway's Path to Product
10/1/2025 - MCP Gateway Status Update - October 1st 2025

💥 MVP

MCP Gateway — MVP Write-up

Executive summary

Deliver a minimal, lovable MCP Gateway on OpenShift that proxies MCP traffic end-to-end, composes "virtual MCP servers," and enforces basic identity-aware filtering and authorization while emitting per-tool telemetry

Outcomes and success criteria

Primary outcomes

- Agents can connect to one gateway endpoint and reliably use tools across multiple MCP servers
- Platform teams can allow or deny tool access by identity and hide disallowed tools from discovery
- Operators can audit who called which tool and see basic reliability and latency metrics

Success criteria

- P0 workflows: initialize → tools/list → tools/call with streaming SSE complete successfully
- Tools from ≥2 MCP servers composed under one virtual MCP and disambiguated by deterministic namespacing
- Per-tool allow/deny enforced and reflected in tools/list filtering
- Metrics exist for per-tool call counts, error rates, and latency percentiles
- Audit logs produced for initialize, tools/list, and tools/call with caller, tool, and outcome

In-scope capabilities (MVP)

Data plane

- Streaming MCP over HTTP: initialize, tools/list, tools/call with SSE
- Incremental JSON-RPC inspection and routing without full-body buffering
- Session stickiness by session ID for a single gateway replica

Virtual MCP Servers

- Compose tools from multiple MCP servers into a single logical endpoint
- Deterministic namespacing and collision avoidance
- Per-virtual-server policy bundle (authZ rules, filters, visibility)

Filtering

- Identity-aware filtering on tools/list so callers only see allowed tools
- Optional parameter-level input filtering for tools/call (simple patterns)

Authorization

- Basic allow/deny per tool mapped to caller identity
- Token validation at edge for bearer/API key, with clear upgrade path to OIDC

Tool management (MVP slice)

- Register MCP servers with URL, auth method, and health check
- Tag and select tools for inclusion in a virtual MCP
- Secrets binding for upstream credentials stored via Kubernetes secrets

Metrics, audit, and tracing

- Per-tool metrics: calls_total, errors_total, latency histograms
- Session metrics: active sessions and stream durations
- Structured audit logs for initialize/list/call including identity and outcome
- Basic OpenTelemetry spans per tools/call

Plugins / processors and guardrails (MVP slice)

- Pluggable request/response processor chain
- Prompt poisoning detection via regex pattern set with block or mask action
- Ability to mark guardrail policies at tool or virtual MCP scope

Delivery and ops

- Envoy/Istio gateway with ext_proc for JSON-RPC inspection and routing
- RHCL/Kuadrant alignment for future ext-auth and rate limits
- CRDs and samples for GitOps flows

Deliberately out of scope (defer to R1+)

- Resumable sessions and cross-replica stickiness
- Fine-grained policy engines (OPA/Cedar) beyond allow/deny lists
- Token-based quotas and budgets
- Advanced guardrails using semantic classifiers
- Multi-replica scale, circuit breaking, and outlier detection
- Registry and developer portal UX beyond minimal YAML and examples
- A2A protocol routing
- stdio backends and gateway-managed backend lifecycle

Architecture approach

Short-term implementation

- Istio Ingress/Gateway running Envoy
- ext_proc gRPC service implements incremental JSON-RPC parsing, tool extraction, and routing
- Single replica for MVP to simplify session stickiness
- Controller watches CRDs and programs routing, policies, and health

Target evolution

- Migrate JSON-RPC logic to Proxy-Wasm for sandboxing and RHCL delivery
- Adopt Envoy stateful-session primitives as they mature
- Integrate Kuadrant ext-auth and rate-limit descriptors

Interfaces and CRDs

CRDs

- MCPServer
    - spec.url, spec.auth (bearer, apiKey), spec.healthProbe
    - status.health, status.tools
- VirtualMCP
    - spec.servers[] with tool selectors and deterministic namespaces

    - spec.policies.toolSelection, spec.filters, spec.visibility

- ToolPolicy

    - spec.allow[] and spec.deny[] mapped to identities, groups, or service accounts
    - spec.guardrails (regex rules, action=block|mask), spec.auditLevel

External interfaces

- MCP client connects to VirtualMCP hostname
- Operators manage via CRDs and sample kustomize overlays

Non-functional requirements

- P50 added latency ≤ 5 ms per tools/call through gateway on reference hardware
- No full-body buffering on streams larger than 1 MB
- Graceful degradation: if a backend is down, only its tools are hidden and other tools remain usable
- All policy changes are audited and applied within 10 seconds

Deployment considerations

- Supported footprints: self-managed and managed OpenShift, classic and HCP, multi/compact/SNO, connected and restricted networks, x86_64 and aarch64
- Shipping as an Operator with CRDs, example kustomize overlays, and sample dashboards
- Minimal UI for MVP: YAML-first with oc/kubectl examples and a small docs site

Metrics, logs, and dashboards

Metrics

- mcp_tool_calls_total{tool,virtual,server,outcome}
- mcp_tool_call_duration_seconds_bucket{tool,virtual,server}
- mcp_sessions_active{virtual} and mcp_stream_bytes_total{direction}

Logs

- Structured JSON with requestId, sessionId, identity, tool, policyDecision, latencyMs, outcome

Dashboards

- Top tools by volume and error rate
- Latency heatmap by tool and backend server
- Audit search by identity and tool

Risks and mitigations

- **Auth maturity**: If OIDC wiring slips, support API keys and bearer tokens with clear upgrade docs
- **Streaming complexity**: ext_proc adds overhead; keep processor logic minimal and benchmark early
- **Spec churn**: Pin to a specific MCP spec version and maintain conformance tests
- **Timeline pressure**: Prioritize VirtualMCP, filtering, basic authZ, and metrics ahead of UI polish

Deliverables checklist

- Envoy/Istio gateway with ext_proc service implementing incremental JSON-RPC routing
- Controller and three CRDs: MCPServer, VirtualMCP, ToolPolicy
- Per-tool metrics, structured audit logs, and example dashboards
- Guardrail processor with regex rules and block/mask actions
- Reference examples: register external MCP, compose a VirtualMCP, apply ToolPolicy, run e2e agent call
- Documentation: install, CRD reference, troubleshooting, performance guidance

MVP demo plan

- Live e2e: agent connects to VirtualMCP, lists tools from two backends, calls one tool and streams result
- Show list-time filtering by switching identity and re-listing tools
- Trigger guardrail block to demonstrate policy and audit
- Open dashboard to show per-tool metrics and a recent audit entry

Cut lines if needed

- If parsing performance risks the date, constrain processors to list-time only and defer response processors

- If metrics work slips, deliver calls_total and error rate first, add latency histograms in a patch
- If secrets UX slips, restrict to bearer token and API key, document rotation steps, add OIDC in R1

Roadmap

MCP Gateway Roadmap

MVP — Developer Preview

Goal: Make MCP traffic work end-to-end on OpenShift with basic control, composition, and visibility

Architecture

- Envoy/Istio Gateway with ext_proc for JSON-RPC inspection and routing
- Stateless data plane, small controller to manage backends and virtuals
- CRDs: MCPServer, VirtualMCP, ToolPolicy

Must-have features

- Streaming MCP proxy for initialize, tools/list, tools/call with SSE
- Virtual MCP Servers that compose tools from multiple MCP servers
- Deterministic tool namespacing to avoid collisions
- Identity-aware filtering on tools/list so callers only see allowed tools
- Basic authorization via allow/deny per tool mapped to caller identity
- Backend registration for in-cluster and external MCP servers with health checks
- Auditable logs per method call with caller, tool, outcome
- Metrics per tool call: count, error rate, p50/p95 latency

Non-functional bars

- Low-overhead streaming path without full-body buffering
- Session stickiness by session ID for a single gateway replica
- Clear failure behavior and safe degradation when a backend is down
- Docs and samples: bring your own MCP, create a virtual MCP, call from an agent

Deliberate defers from MVP

- Resumable sessions and cross-replica stickiness
- Fine-grained policy engines (Cedar/OPA) beyond simple allow/deny
- Token-based rate limits and budgets
- Guardrail processors beyond simple regex redaction

- UI beyond basic YAML and kubectl examples

Release 1 — Hardening & Policy (post-MVP)

Goal: Turn control into governance, reduce toil, and prep for scale

Data plane

- Proxy-Wasm implementation of JSON-RPC parsing to replace/ext_proc where practical
- Stateful session filter adoption for stickiness, retry where safe

AuthN/Z and filtering

- OIDC bearer validation at the edge with claim mapping
- Pluggable authZ via ext-auth (Kuadrant/Authorino) with ToolPolicy CRD as source of truth
- List-time filtering + call-time enforcement remain consistent

Rate limiting and quotas

- Per-tool and per-caller request rate limits via Kuadrant descriptors
- Concurrency caps per backend tool

Observability

- OpenTelemetry spans per tools/call with backend spans
- Default dashboards for top tools, error hotspots, tail latency

Guardrails

- Processors chain with regex/PII redaction on responses
- Mark tools with guardrail policies and enforce block or mask actions

---

Release 2 — Registry, Catalog, and UX

Goal: Make discovery and composition first-class and self-service

Registry/Catalog

- Import MCPServer metadata from registry
- Publish VirtualMCP entries into enterprise sub-registry
- Sync health and versions back to registry metadata

Tool Management

- Ownership, risk tier, cost tags, and lifecycle states
- Version pinning and diff views for tool changes

UX

- Console plugin for registering servers, composing virtual MCPs, assigning policies
- Developer "playground" to list tools and run test calls with traces

---

Release 3 — Scale, Resilience, and Cost

Goal: Operate confidently under load and control spend

Scale and reliability

- Multi-replica gateway with cross-replica session affinity
- Circuit breaking and outlier detection per backend
- Partial degradation that hides or greys out unhealthy tools

Advanced limits

- Token-based rate limiting and budgets per tool and team
- Priority and fairness across tenants

Cost and analytics

- Cost annotations per call where known or estimated
- Budget alerts and usage reports by team, tool, and environment

---

3.next — Advanced Guardrails, A2A, and Ecosystem

Goal: Deep governance and broader agent connectivity

- Semantic guardrails via external classifiers and webhook approval flows
- OPA or Cedar policies for context-aware decisions on tools/call
- Resumable sessions across restarts with external session store
- Optional A2A routing patterns where needed
- Interop tests against popular gateways and conformance suites

---

Cut-lines if time gets tight

- If parsing in Wasm isn't ready, keep ext_proc for MVP and focus on latency tests
- If OIDC wiring slips, ship API keys for MVP and upgrade to OIDC in Release 1
- If UI slips, ship CRDs + docs and push Console plugin to Release 2

Why this shape works

- Delivers the core promise fast: virtual MCP, filtering, basic authZ, and visibility
- Uses Envoy/Istio you already operate, minimizing bespoke infra
- Leaves heavy lifts (sessions at scale, policy engines, token budgets, UI polish) to clearly defined follow-ons

If you want, I can turn this into an epic → feature → story backlog with acceptance criteria and CRD sketches for MCPServer, VirtualMCP, and ToolPolicy next.

🌱 Possible Features

MCP Gateway List of Possible Features

Core data plane

- Native MCP over HTTP with streaming (initialize, tools/list, tools/call, SSE)
- Incremental JSON-RPC parsing and routing without full-body buffering
- Session handling with sticky routing by session ID
- Backend MCP server discovery and registration (in-cluster and external)
- Transport security with TLS termination and mTLS to backends
- High availability with stateless data plane and shared control metadata

Virtual MCP Servers

- Compose multiple backend MCP servers into a single logical endpoint
- Tool name de-duplication and deterministic namespacing
- Per-virtual-server policy bundles for authN/Z, rate limits, and filters
- Versioned compositions with change review and rollback
- Scoped visibility of virtual servers by tenant, project, or team

Filtering

- Identity-aware tool list filtering on tools/list
- Parameter-level input filtering for tools/call
- Output filtering with redaction and field-level masking
- Conditional filters by environment, user attributes, or tool metadata
- Allow/deny lists for methods, parameters, and target hosts

Authorization

- Authentication via OAuth 2.1 / OIDC tokens, API keys, or mTLS identities
- Per-tool RBAC with roles, groups, and attributes
- Policy engine integration (e.g., OPA or Cedar) for context-aware decisions
- Decision-time enrichment with request context and registry metadata
- Audit-enforced deny reasons and policy evaluation traces

Tool Management — what fits here

- **Registration**: add MCP servers with URL, capabilities, auth method, health probes
- **Cataloging**: store and tag tools with owner, risk level, data classification, cost tier
- **Curation**: assemble "tool belts" for projects or virtual servers
- **Versioning**: pin to server versions, compare tool diffs, staged rollouts
- **Health & status**: heartbeat checks, readiness, brownout indicators
- **Quotas & budgets**: per-tool call limits, monthly budgets, cost center tagging
- **Transformations**: request/response transforms, schema adapters, parameter templating
- **Sandboxing**: mark tools as experimental with scoped access and stricter limits
- **Deprecation**: lifecycle states with sunset dates and automatic warnings
- **Ownership & approvals**: change control with approvers and required reviews
- **Metadata sources**: sync from Registry and annotate locally
- **Secrets binding**: bind credentials per tool or per caller with rotation policies

MCP / AI metrics and observability

- Per-tool counters, rates, latency histograms, error codes
- Session metrics including active sessions and stream durations
- Token and payload size metrics where available
- OpenTelemetry traces for each tools/call with backend spans
- Cost indicators per call when known or estimated
- SLOs and alerting for failure rate, latency, and saturation
- Correlated audit logs with caller, tool, parameters metadata, and outcomes

Plugins / processors and guardrails

- Pluggable request and response processors chain
- Prompt poisoning detection on inputs with configurable severities
- PII/secret scanning and redaction on outputs
- Toxicity and safety classifiers with policy-driven block or review
- Domain guardrails, e.g., disallow external email recipients or raw SQL to prod
- Marking and enforcing guardrail policies per tool, per virtual server, or global
- Moderation webhooks for external approval flows
- Quarantine mode to capture samples for investigation without delivery to agent

Rate limiting, quotas, and fairness

- Per-tool and per-caller rate limits
- Token-based limits (tokens per minute) and burst control
- Concurrency caps per backend tool to protect capacity
- Priority tiers and fair-share scheduling across teams
- Budget enforcement with soft warnings and hard stops

Reliability and resilience

- Circuit breaking and outlier detection per backend MCP server
- Retries with idempotency keys and backoff where safe
- Partial degradation modes that hide unhealthy tools automatically
- Backpressure on streams to prevent gateway overload

Platform and multitenancy

- Namespace and project scoping for tools and virtual servers
- Policy inheritance with overrides by tenant or environment
- NetworkPolicies and egress controls for external tools
- Secrets management and rotation for upstream credentials

Developer and operator experience

- CRDs or API for declaring servers, tools, policies, and virtual MCPs
- UI for composition, policy assignment, metrics, and live debugging
- Developer “playground” to list tools and run test calls with trace views
- Dry-run and diff views for policy and composition changes
- Templates and quickstarts for common MCP servers

Registry, catalog, and lifecycle

- Import MCP server metadata from a registry
- Publish virtual MCP servers to an enterprise sub-registry
- Integrity checks, signing, and attestation for tool sources
- Lifecycle states with promotion from dev to prod via GitOps

Interop and standards

- Conformance to current MCP spec including OAuth 2.1 and streaming
- Optional support for A2A routing where applicable
- Compatibility tests against popular gateways and MCP servers

Governance and compliance

- Centralized audit trail per method call with immutable storage option
- Data classification tags and location constraints per tool
- Policy packs for regulated environments and evidence export

Performance and capacity

- Zero-copy streaming paths where possible
- Horizontal scaling with sharded session affinity
- Benchmark suite for latency, throughput, and tail performance

MCP Servers

🤝 ISV MCP Servers for 3.0

Partnership Ecosystem - ISV MCP Servers for 3.0

The **Partnership Ecosystem team** is working toward bringing in at least 15 MCP servers from our partner ISVs as part of the v3.0 release. This team is responsible for driving these integrations and coordinating with partners. These MCPs will not be certified or supported by Red Hat, but they are "blessed" — meaning recommended for experimentation and evaluation.

ISV MCPs

ISV / Partner	General Use & Value Proposition (Hypothetical / Possible)
Couchbase	Enables agents to interact with Couchbase clusters (queries, schema introspection, data operations).
CrowdStrike	Surfaces Falcon platform data (detections, incidents, behaviors) for contextual security reasoning.
CyberArk	Could expose privileged access vaults, secrets management, or credential policies.
Dynatrace	Likely to offer telemetry, performance metrics, traces, and root cause context.
Elasticsearch	Provides search and aggregation over indexed documents and datasets.
HashiCorp	Could expose Terraform state, Vault secrets, or Consul service discovery.
IBM Wxflows	May allow agents to connect with workflow/automation functions.
JFrog	Potential access to artifact repositories, dependency metadata, and builds.
Mulesoft	Could support API discovery, orchestration flows, and endpoint invocation.

ISV / Partner	General Use & Value Proposition (Hypothetical / Possible)
Oracle / Rubrik	Expected to cover database access, schema queries, backup/recovery operations.
Palo Alto Networks	Likely to provide firewall logs, network policy, and security alerting.
Redis / Redis Cloud	Provides key-value operations, caching, and fast in-memory data queries.
Snyk	Surfaces vulnerability scans, dependency graphs, and remediation info.
Unstructured	Could offer ingestion, parsing, and structuring of unstructured content.

Considerations & Disclaimers

- These MCPs will be listed on a **public website** for visibility. They will not appear in the Red Hat registry or catalog until those features are available in the 3.x lifecycle.
- These are **"blessed" recommendations**, not certified or supported by Red Hat. Usage should be considered exploratory.
- For many partners, the exact APIs and capabilities are still unknown — descriptions here are **general expectations** based on the vendor's domain.
- Interfaces and stability may evolve, especially as MCP implementations are still maturing.
- Security and access control will be important — expect role-based access, auditability, and isolation requirements.
- The **evaluation process is under development** and will be built out as the protocol stabilizes.

🌟 Red Hat Product MCP Servers

Possible Red Hat Product MCP Server - Stack Rank

🔝 Top Tier (High Value)

Red Hat OpenShift (OCP)

- **Why**: As the foundation for containerized workloads and Kubernetes orchestration, OCP offers critical runtime, deployment, and infrastructure metadata.
- **Use Case**: Use an MCP server to expose pod status, configmaps, workloads, and namespaces to help agents diagnose issues, perform impact assessments, or automate day-2 operations.

Ansible Automation Platform (AAP)

- **Why**: AAP orchestrates repeatable IT tasks and workflows. It has access to execution logs, playbooks, inventories, and system states.
- **Use Case**: Enable AI agents to summarize playbook executions, suggest changes, or auto-generate workflows from intent or past actions.

Red Hat AI Assets on OpenShift AI (RHOAI)

- **Why**: Central to building, deploying, and governing LLMs and agents—MCP is already core here.
- **Use Case**: Internal registry of models, agents, guardrails—already aligns with GenAI work.

🧭 Mid Tier (Valuable but More Specialized)

Red Hat Insights

- **Why**: It already collects telemetry, logs, and risk data—perfect context to feed into LLMs for recommendations, automation, or reporting.
- **Use Case**: Security posture assistant, drift analysis summaries, automated remediation reasoning.

Red Hat Enterprise Linux (RHEL)

- **Why**: It's the OS layer for nearly every enterprise deployment. Telemetry, configuration, patch state, and compliance data make RHEL a foundational context provider.

- **Use Case**: Assistants can analyze kernel logs, patch status, or system health to recommend remediations, upgrades, or changes.

🧩 Lower Tier (Situational or Specialized Context)

Red Hat Advanced Cluster Management (ACM)

- **Why**: Multi-cluster, hybrid cloud control—ideal for global context and policy-based management.
- **Use Case**: Generate cluster state summaries, policy diff reports, or AI-driven policy suggestions.

Red Hat Satellite

- **Why**: Lifecycle management, patching, and system state data can provide valuable configuration context.
- **Use Case**: Use MCP to pull server states and create change plans or validate deployments via agentic workflows.

Red Hat Quay (Container Registry)

- **Why**: Rich metadata on image security, vulnerabilities, and usage.
- **Use Case**: LLM agents doing software supply chain risk analysis or policy compliance checks.

Red Hat Documentation MCP (Integrate with latest official Documentation for all products)

🤔 Possible Initial MCP Servers

Possible Initial MCP Servers for MVP

These MCP servers are live today, actively maintained, and already used in production proofs-of-concept—so they can provide immediate, demonstrable value in the MVP.

✅: Tested Successfully in RHOAI

Category	MCP Server (exists today)	Why it matters for MVP
Kubernetes & InfraOps ✅	**Kubernetes MCP Server –** mcp-kubernetes-server (v0.1.11, released May 11, 2025) is a Python-powered server that translates natural language into kubectl actions and provides cluster introspection to agents. (pypi.org)	Gives AI agents the ability to query pod health, describe resources, or perform dry-run actions across OpenShift or Kubernetes clusters.
ChatOps / Collaboration	**Slack MCP Server** – MIT-licensed server that lets AI agents post, read threads, DM users, and trigger Slack workflows; supports stdio + SSE, proxy mode, and fine-grained token scopes. (github.com)	Instant DevOps productivity: an assistant can triage alerts, summarize channels, or file tickets without leaving Slack.
IT Service Management ✅	**ServiceNow MCP Server** – open-source repo **and** certified Store app; AI can query, create, or update incidents, change requests, catalog items, etc., with full OAuth support. (github.com, store.servicenow.com)	Automates ticket triage, change-management chatbots, and "one-shot" incident creation from natural language.

Category	MCP Server (exists today)	Why it matters for MVP
CRM / Customer Support	**Salesforce MCP Server** – CLI-installable server that exposes SOQL querying, record CRUD, Apex code access, and schema introspection. (github.com)	Lets support or sales assistants pull account context, open cases, and update opportunities directly from an AI prompt.
Observability & SecOps	**Splunk MCP Server** – FastMCP-based tool that runs SPL queries, returns logs/metrics, and auto-scrubs sensitive data. (github.com)	Enables an AI SRE bot to explain spikes, correlate incidents, or draft a post-mortem using live Splunk data.
	Dynatrace MCP Server – official Dynatrace-OSS project exposing DQL queries, problem feeds, and vulnerability data. (github.com)	Gives agents real-time service health, letting them recommend rollbacks or capacity fixes inside OpenShift.
Developer Productivity ✅	**GitHub MCP Server** – GitHub-maintained server for listing repos, issues, PRs, commits and creating comments/branches. (github.com)	Fuels coding copilots that can open PRs, draft release notes, or review diffs while respecting repo permissions.
Structured Data / Regulated Verticals	**PostgreSQL MCP Server** (@modelcontextprotocol/server-postgres) – read-only SQL querying with schema discovery, run in a container or as a Node service. (npmjs.com)	Ideal for healthcare/finance use-cases that need tight RBAC, audit trails, and deterministic queries against clinical or financial databases.
Cross-App Automation	**Zapier GitHub MCP (Zapier MCP Beta)** – hosted server that unlocks 7,000-plus SaaS actions via Zapier without writing glue code. (zapier.com)	"Swiss-army-knife" for quick PoCs: one endpoint gives agents access to calendars, Jira, NetSuite, etc., under Zapier's enterprise security model.

Kubernetes MCP Servers

Kubernetes MCP Servers

Below are 3 Kubernetes MCP Server options with comparisons

- **k8s-mcp-server**
  - GitHub: reza-gholizade/k8s-mcp-server (GitHub, GitHub)
- **mcp-kubernetes-server** (Python/Docker)
  - GitHub: feiskyer/mcp-kubernetes-server (GitHub)
- **manusa/kubernetes-mcp-server**
  - GitHub: manusa/kubernetes-mcp-server (GitHub)

Side-by-Side Comparison

Criteria	k8s-mcp-server	mcp-kubernetes-server	manusa/kubernetes-mcp-server
Language	Go	Python (>=3.11)	Go
Deployment	Single binary (stdin/stdout)	Python/Docker, requires kubectl & helm	Single binary (also npm/Python/Docker options)
Dependencies	None	kubectl, helm, Python runtime	None (uses K8s API directly)
CRUD support	Read-only (list/describe/logs)	Full kubectl via subprocess	Full native CRUD + exec, metrics, helm
Helm support	No	Yes (via subprocess)	Yes (native helm integration)
Streaming	JSON-RPC stdio	stdio/SSE/HTTP transports	HTTP/SSE streaming

Criteria	k8s-mcp-server	mcp-kubernetes-server	manusa/kubernetes-mcp-server
Performance	Very low-latency	Moderate (subprocess overhead)	Low latency, high performance
Platform	Cross-platform	Containerized or local Python	Cross-platform binary

---

1. **k8s-mcp-server** (Go, stdin/stdout JSON-RPC)

- **Language & Deployment**: Native Go binary; minimal footprint, standalone.
- **Features**:
  - Discover API resources at runtime.
  - Supports listing, describing, and logs via JSON-RPC over stdin/stdout.
- **Pros**:
  - Lightweight, low-latency, no external deps.
  - Great for embedding in Go-based AI agents and automation.
- **Cons**:
  - Limited to basic and read/log operations.
  - Doesn't support write/helm/exec by default.
- 🔧 **Best for**: Go ecosystems; building custom tools where you control behavior directly.

---

2. **mcp-kubernetes-server** (Python/Docker v0 .0 .11 – May 11 2025) (GitHub, Medium, LobeHub)

- **Language & Deployment**: Python (requires >=3.11) and Docker.
- **Features**:
  - Acts as bridge to AI tools (Claude, Copilot, Cursor).
  - Executes kubectl and helm via subprocess.
  - Supports natural language → kubectl/helm, cluster queries, diagnostics.
- **Pros**:
  - Flexible; fits into any Python/Docker pipeline.
  - Easy to deploy via Docker (no Go compile required).

    - Rich toolset: command execution, helm, write/delete toggles, HTTP/SSE transport options (PyPI).
- **Cons**:
    - Heavier runtime and external tool dependencies (kubectl, helm).
- 🔧 **Best for**: Python-centric stacks or containers; natural-language interactions with full kubectl/helm support.

---

3. **manusa/kubernetes-mcp-server** (Go, full native API support) (GitHub)

- **Language & Deployment**: Go with native binary for Linux, macOS, Windows, plus npm, Python wrapper, Docker.
- **Features**:
    - Full CRUD support on K8s/OpenShift resources.
    - Pod exec, logs, top metrics, namespace and event fetches.
    - Helm: install/uninstall/list releases.
    - Native Kubernetes API integration—no shell tools needed (GitHub).
    - Config hot-reloading, SSE/HTTP streaming, extensive testing suite (GitHub).
- **Pros**:
    - Comprehensive, performant, low latency.
    - One binary, fully featured across platforms.
- **Cons**:
    - Bigger binary; config complexity.
    - Slightly slower innovation cadence (latest release July 2, 2025) .
- 🔧 **Best for**: Production-grade needs when full control over resources, metrics and Helm is essential.

Which Should You Pick?

- **Go enthusiast / minimal setup** → **k8s-mcp-server** is perfect for lightweight builds.
- **Python/Docker with natural-language tools** → go with **mcp-kubernetes-server**, full kubectl & helm in containers.
- **Need full Kubernetes and Helm features with performance** → **manusa/kubernetes-mcp-server** is the richest and most robust option.

😮 Vulnerabilities in the News

MCP Issue News Stories

Listing of news stories related to MCP vulnerabilities

Severity	Date	Title & Link	Summary	Why it matters	Mitigation (General)
Critical	21 Oct 2025	MCP prompt-hijacking attack (oatpp-mcp) – The Register	Predictable session IDs in the Oat++ MCP implementation allowed attackers to hijack live agent sessions and inject malicious responses. (JFrog)	Shows protocol-layer session handling can let attackers take over live agent conversations, not just the LLM prompt.	Generate session IDs u cryptographically secur tokens (≥128 bits), avoi enforce session expiry, event IDs, enforce TLS.
Critical	7 Oct 2025	RCE in Framelink Figma MCP server (CVE-2025-53967) – Imperva	RCE within a widely used Figma MCP server integration.	Third-party tool servers can breach agent sandboxes and escalate access on host machines.	Apply vendor patch, run least privilege, restrict n access and tool invocat container sandbox with capabilities.
Critical	13 Jun 2025	MCP Inspector RCE (CVE-2025-49596)	Dev-tool web UI lacked authentication, enabling an attacker on same network or via malicious page to execute code.	Developer machines and dev tools are often overlooked yet become easy entry points for production compromise.	Upgrade to patched ver to localhost only, require authentication tokens, d web-UI in production, e network isolation.
High	18 Jul 2025	Command injection in `mcp-package-docs` server (CVE-2025-54073)	Input validation flaw allowed OS-level command execution in a documentation-tooling MCP server.	Popular doc/data servers integrated into MCP stacks can become remote shells if inputs aren't sanitized.	Update to patched com non-root user, drop unn capabilities, apply AppArmor/SELinux, val all inputs.

Severity	Date	Title & Link	Summary	Why it matters	Mitigation (General)
High	4 Aug 2025	EscapeRoute flaws in Filesystem MCP server – Cymulate	Two vulnerabilities allowed sandbox escape, unrestricted file access, and potential code execution.	Filesystem MCPs directly bridge agent → file system and thus can expose host or cluster storage.	Use read-only mounts, allowlists, restrict execu privileges, isolate conta seccomp profile.
High	29 Sep 2025	Fake "Postmark MCP" npm server stole emails – The Hacker News	A malicious npm package (typosquatted) acted as an MCP server and exfiltrated emails via BCC.	Supply-chain risk: a malicious MCP server package integrated downstream can compromise many users.	Pin to verified publisher signatures, use SBOM checks, restrict package rotate exposed credenti outbound email flows.
High	8 Sep 2025	MCP Inspector XSS (CVE-2025-58444)	XSS via malicious redirect URIs in dev tool triggered host command execution.	Even UI-facing dev tools can pivot into full host compromise if web endpoints are insecure.	Update to patched vers redirect URIs, enforce C same-site cookies, isola network.
Medium	15 Sep 2025	Identity fragmentation in MCP ecosystems – TechRadar Pro	Non-CVE analysis: fragmented identity/credential management across MCP servers increases lateral-access and insider risks.	Poor identity linkage means a single compromised credential can pivot broad access.	Centralize identity, adop access, remove static s enforce per-tool scopes per-server authZ.
Medium	25 Jun 2025	Anthropic won't fix SQLi in SQLite MCP server – The Register	Unpatched SQL injection in example/reference server provided by vendor; vendor recommended avoiding rather than fixing.	Example/reference servers may be unsafe for production — they become de facto defaults wrongly.	Do not use sample serv production; fork and pat parameterized queries; privileges; implement re users.

Severity	Date	Title & Link	Summary	Why it matters	Mitigation (General)
Informational	3 Oct 2025	MCP stacks 92 % exploit probability study – VentureBeat	Research shows exploit likelihood increases with number of MCP plugins/tools — “10 plugins => 92 % probability”.	Quantifies the risk scaling with tool count; warns of complexity-driven insecurity.	Limit number of active M servers/tools, segment runtime monitoring for a tool invocation.

📚 Learning

🧱 Foundational Knowledge

These are the pillars of what is being built

1. Model Context Protocol (MCP)

- Understand the **MCP spec** (pre-1.0 state, goals, transport types: HTTP, WS, stdio, SSE)
- MCP server/client behaviors, use of tools, metadata schema (you already have it)
- Registry vs. Catalog: differences in purpose, scope, and lifecycle

2. OpenShift AI & Operator Pattern

- OpenShift operators: how they manage lifecycle, why they're used
- Familiarize yourself with OpenShift AI as a deployment and orchestration platform for MCP-based services
- Understand **how RH packages enterprise capabilities on top of upstream tech** (OpenShift vs. Kubernetes → MCP vs. upstream)

3. Security Building Blocks

- RBAC (Kubernetes-based): especially around scoped resource access
- OAuth 2.0: token-based access delegation
- API Gateways (e.g., **3scale**, **Envoy**, **Kong**): filtering, rate-limiting, authorization use cases

---

🧠 Strategic Engineering Topics

These are concepts that came up repeatedly and will shape architecture decisions.

4. Service Discovery Models

- Static vs. dynamic discovery (with MCP registries)
- Registry scoping (cluster-wide vs. namespace vs. tenant)
- Tool "views" for agents — how to expose a filtered set of capabilities

5. Sandboxing and Quarantine

- OpenShift Sandboxed Containers / Kata containers
- Patterns for **quarantine + certify + promote** (like OperatorHub certification pipeline)
- Tool poisoning / rock-pooling security concepts

6. Lifecycle & Deployment Patterns

- Managing shared MCP servers across clients
- How upstream Anthropic registry implements intake and what Red Hat could improve
- How to do upgrades, versioning, deprecation of MCP servers at scale

---

📚 Suggested Learning Format

Area	Learning Resource Type
MCP & Registry	Read Anthropic's reference docs + Red Hat internal notes/spec
RBAC + OAuth2	Kubernetes RBAC docs + Auth0 / Okta OAuth2 guides
Sandboxing	OpenShift Sandbox docs
API Gateway	FastAPI, Kong or Envoy official docs, 3scale patterns
Operators	OperatorHub & RH Operator Certification docs
Partner Tech	Blog posts + partner product pages (Stacklok, Anthroopic, others)

🔥 Questions to Drive Clarity in Strategy Meetings

- How will we version and certify MCP servers?
- What makes our registry more trustworthy than upstream?
- How do we ensure discovery doesn't become a vulnerability?
- What are must-have security/compliance features for enterprise consumption?

Demos

MCP Demo Sheet Listing

MCP Registry Related Demos:

Daniele Martinoli's work: MCP Catalog POC

- **Presentation Slides -** MCP Catalog POC – Google Slides
- **GitHub Repository -** RHEcosystemAppEng/mcp-catalog-operator

Vaibhav Jain's work: Created a generic Kubernetes Operator

- GitHub repo: github.com/btc-hackathon/mcp-operator
- Catalog image: quay.io/vajain/mcp-operator-catalog:v0.0.7

Kun's work : GUI for MCP registry

- https://uxe-mcp-registry-kyan-28b8378727937d47489a627f0cd104a6e20d33a12.pages.redhat.com/
- https://gitlab.cee.redhat.com/kyan/uxe-mcp-registry

Maor's work: REST API for MCP registry

- https://mcp-registry-poc.devshift.net/v0/servers
- https://gitlab.cee.redhat.com/ai/mcp-registry

👩‍❤️‍👨 Stakeholders

Stakeholders

Last Updated: 2025-08-26

AI Business Unit

- Joe Fernandes - Vice President & GM, AI Business Unit - Informed
  - Tushar Katarki - Senior Director, Product Management Global - Informed
  - Jeff DeMoss - Director, AI Product Management
    - Peter Double - Principal Product Manager - In charge of all MCP Product Management - Responsible for MCP Registry (Project Leader)
      - Main Product Manager Responsible for delivery of this product
    - Adam Bellusci - Senior Principal Product Manager - Responsible for Model Registry and AI Hub, Consulted for MCP Registry
    - Adel Zaalouk - Principal Product Manager - Llama-Stack MCP Tooling - Concerned with Registry Success - Consulted for MCP Registry
    - Rob Greenberg - AI Hub Lead
    - Jenny Yi - RHOAI Dashboard Lead

RHOAI Engineering Teams

- **Crimson Team** - Gen AI UI Development, RHOAI Dashboard - Developing both the playground and AI Available Assets
  - Darach Cawley - Team Lead
- **Agentic/MCP Team** - Own all Agentic / MCP related product work
  - Justin Sun - Product Owner
  - Bill Murdock - Team Lead
- **RAG Team** - RHOAI Document Ingestion and Retrieval Functionalities
  - Francisco Arceo - Product Owner
- **Llama-Stack Core Team** - Interested in Tool interactions with Llama Stack
  - Roland Huß - Lead LLS Architect

User Experience

- Daniel Warner - Developing UI Prototypes for MCP interactions

- Danielle Jett - Developing UI Prototypes for Playground and AI Available Assets (AAA)
- Jenn Giardino - Responsible for User Experience related to MCP Catalog/Registry

Engineering - General

- Steven Huels - Vice President, AI Engineering - Overseeing all AI Products - Informed
    - Jason Greene - AI Engineering Architect - Accountable for MCP Registry Delivery
    - Jessica Forrester - AI Engineering Architect - Accountable for MCP Registry Delivery
    - Catherine Weeks - RHOAI Engineering Director - Accountable for Delivering MCP Capabilities
        - Ann Marie Fred - Senior Principal Software Engineer - Responsible for MCP capabilities - rolls up to Catherine
    - Sherard Griffin - Senior Director, Software Engineering Global - Informed
        - Jen Albertson - Director - everything across RHOAI - Informed
            - Platform Dashboard
            - MCP & Model - Registry/Catalog
            - Dana Gutride - Senior Manager, Engineering - Accountable for UI related to registry
                - In charge of all UI Engineers related to this work
            - Jay Koehler - senior manager for RHOAI - rolls into Jen Albertson - Consulted
                - Partnering with Ramesh on this
                - Observability - Platform (Core)
                - Ramesh Reddy - Manager, Engineering - Reports in to Jay Koehler
                    - MCP Registry - Accountable
            - Lindani Phiri - Senior Principal Software Engineer - Architect - rolls into Jen Albertson - Consulted
                - Anything under Jay
                - Security
                - Observability
                - Workload Management
                - (Infrastructure team)
            - Daniele Zonca - Senior Principal Software Engineer - Architect - rolls into John Graham - Consulted

        - Serving - llm-d (serving orchestration)
        - Trustry.ai
        - Model Registry
        - Jounce
    - Kevin Howell - Principal Software Engineer - Observability - Consulted

Other Interested Parties

- Jason Smith - Manager, Software Engineering - Responsible for MCP strategy for XE (Experience Engineering) - Consulted for MCP Registry
- Idan Levi - Senior Manager, Software Engineering - informed
    - Josh Salomon - Senior Principal Software Engineer - Developing strategy around MCP Registry. Microsoft Contact for possible collab on upstream registry - Consulted
    - Yuval Kashtan (US) - Senior Principal Software Engineer - Security Input - Consulted
- Ilan Pinto - Manager, Software Engineering - Informed
    - Daniele Martinoli - Principal Software Engineer - Developing prototypes for MCP work and moving forward catalog work - Consulted
- Maor Friedman - Principal Software Engineer - Looking to support the Registry Buildout where he can - Consulted
- Vaibhav Jain - Senior Solution Architect - Developed Operator for MCP, Interested in helping build the registry
- Daniel Messer - Senior Manager, Product Management_Global - Operator Hub - Interested in how operators will register MCP servers - Consulted

Links

🚨 Main Downstream Registry Repo

Last Updated: 2025-10-01

Downstream Red Hat MCP Registry* - https://github.com/redhat-ai-tools/mcp-registry

*Current Main Repo for Registry Developement

Red Hat MCP Links

- Collection of Red Hat Recommended MCP Servers - https://github.com/redhat-ai-tools/
- Kubeflow Model Registry - https://github.com/kubeflow/model-registry
- Red Hat Jira MCP Server: https://github.com/redhat-ai-tools/jira-mcp-snowflake
- Red Hat Service Now MCP: https://github.com/redhat-ai-tools/service-now-mcp

Anthropic

- MCP Standard Upstream Repo - https://github.com/modelcontextprotocol/
- MCP Registry Repo - https://github.com/modelcontextprotocol/registry

MCP Registries

- Docker - https://github.com/docker/mcp-registry
- Smithery - https://smithery.ai/

MCP Gateways

MCP Gateway Project Documentation
Gateway Evaluation Document

- Solo.io - https://github.com/agentgateway/agentgateway
- Envoy - Being Extended to become an MCP Gateway
- IBM - https://github.com/IBM/mcp-context-forge
- Lunar - https://github.com/TheLunarCompany/lunar

Authorization

- Authorization Validated Pattern: https://validatedpatterns.io/patterns/layered-zero-trust/
- Helm Chart for MCP Auth (Demo): https://github.com/sabre1041/mcp-auth-demos/pull/2

Jira MCP Tickets

🎟️ Jira MCP Tickets

Last Updated: 2025-08-26

Filter of RHAI MCP Jira Tickets:
https://issues.redhat.com/browse/RHAIRFE-294?filter=12473669#

OCP Networking MCP Gateway Ticket:
OCPSTRAT-2297 - Model Context Protocol (MCP) Gateway

Open Telemetry

Current State of Standards

✔ There is *no existing MCP-specific OpenTelemetry specification*

Across:

- OpenTelemetry community SIGs
- Cloud Native Computing Foundation (CNCF)
- Anthropic's MCP reference implementation
- Stacklok Toolhive (closest to real operational MCP gateways)
- Microsoft MCP clients
- IBM's MCP integrations
- Cursor / Claude usage analytics

➡ **None define MCP semantic conventions.**

There are **standards for RPC**, **HTTP**, **Messaging**, and **Database** spans — but nothing for a protocol like MCP, which is essentially:

- A transport
- A set of tool calls
- A contract between LLM and server
- Optionally: file operations, prompts, config resolution, etc.

This means **Red Hat + Gen-MCP actually get to define the MCP telemetry pattern**, and upstream will follow if it's sane.

What We *Can* Base MCP Telemetry On

The closest analogues:

1. OpenTelemetry Semantic Conventions: RPC

MCP "tool calls" map almost perfectly to RPC semantics:

```
None
rpc.system = "mcp"
rpc.service = "<server name>"
```

```
rpc.method = "<tool name>"
rpc.operation = "<tool name>"
```

2. **HTTP / SSE Transports**

Most MCP servers use:

- HTTP POST for invoke
- SSE for events

So we should reuse:

```
None
http.request.method
http.response.status_code
network.peer.address
```

3. **OpenTelemetry Resource Attributes**

Useful for server identity:

```
None
service.name = "<mcp server name>"
service.version = "<semantic version>"
deployment.environment = "dev/staging/prod"
```

These also map directly to the MCP metadata schema you already have.

A Reasonable MCP Telemetry Definition (Proposed Standard)

You can give this directly to Callum — it's lean, implementable, and follows OTel guidance.

---

1. Span: `mcp.tool.invoke`

When: every time a tool is invoked by the LLM/agent

```
None
Span Name: mcp.tool.invoke
rpc.system = "mcp"
rpc.service = "<server_name>"
rpc.method = "<tool_name>"
rpc.operation = "<tool_name>"
mcp.server.version = "<version>"
mcp.spec.version = "<mcpSpecVersion>"
mcp.tool.input_size = <bytes or char count>
mcp.tool.output_size = <bytes or char count>
mcp.tool.success = true/false
mcp.transport = "http" | "sse" | "ws" | "stdio"
```

---

2. Span: `mcp.server.connect`

When: client establishes a session/connection (SSE/WS/HTTP handshake)

```
None
Span Name: mcp.server.connect
server.address = "<uri>"
network.transport = "tcp"
mcp.transport = "sse" | "ws" | "http"
mcp.server.version = "<version>"
```

---

3. Span: `mcp.server.startup`

When: server boots

```
None
Span Name: mcp.server.startup
service.name = "<server_name>"
```

```
service.version = "<version>"
mcp.spec.version = "<spec version>"
mcp.tools.count = <int>
```

---

4. Span: `mcp.event.emit`

(Optional — for servers using streaming events)

```
None
Span Name: mcp.event.emit
mcp.event.type = "<event_name>"
mcp.event.payload_size = <bytes>
```

Why This Definition Works

✔ Maps 1:1 to Llama-Stack Observability

You told Callum:

> "Telemetry is working in next Llama Stack release — we'll bring tracing and observability in."

Using this proposed definition, both Gen-MCP and Llama-Stack produce **compatible spans**.

✔ Works cleanly with the MCP Registry Metadata

Nearly all fields map back to the metadata schema:

- version
- specVersion
- transports
- authors/maintainers
- categories/tags

This allows registry → tracing correlation out of the box.

✔ Allows Gateway- and Registry-level tracing

This is important because Toolhive + Gateway will eventually need:

- upstream span aggregation
- usage analytics
- trust scoring
- tool-quality evaluation

Are There Any Emerging MCP Telemetry Standards?

After checking:

- Anthropic MCP repos
- Claude MCP RFC discussions
- Stacklok Toolhive issues
- IBM Context Forge MCP instrumentation
- Microsoft's MCP demos
- CNCF agentic working groups
- Cursor's internal telemetry model

➡ **There is no publicly documented MCP OpenTelemetry schema.** ➡ Everyone is improvising or using opaque internal analytics.